Conditionally Linkable Attribute-Based Signatures ⋆

Minh Pham[1] ✉, Khoa Nguyen[1] ✉, Slim Bettaieb[2], Mukul Kulkarni[2], and Willy Susilo[1]

[1] School of Computing and Information Technology, University of Wollongong, Northfields Avenue, Wollongong NSW 2522, Australia
✉ nmp071@uowmail.edu.au, ✉ khoa@uow.edu.au
[2] Technology Innovation Institute, UAE

Abstract. Attribute-Based Signatures (ABS) enable users to authenticate messages under expressive attribute policies while remaining anonymous. Existing ABS variants, however, treat linkability as a static, system-wide property: signatures are either always unlinkable, as in standard ABS, or globally linkable, as in traceable or accountable extensions. This rigid dichotomy fails to capture scenarios where correlation should arise only under explicitly declared conditions.

This work introduces *Conditionally Linkable Attribute-Based Signatures* (CLABS), a framework extending ABS with programmable, context-dependent linkability. Each certified user with attribute x is associated with a linking set L_x over a public context space $\mathcal{T}$. For each context $\tau \in \mathcal{T}$, a public function f_τ specifies how attributes are compared. Two signatures are publicly linkable if and only if $\tau \in L_x \cap L_{x'}$ and $f_\tau(x) = f_\tau(x')$; otherwise they remain unlinkable. This enables selective, verifiable correlation without central trust and with leakage limited to the opt-in bit.

We formalize the syntax and security notions of CLABS, capturing *conditional linkability* and *context-aware anonymity*, thereby ensuring privacy and verifiable linkage under voluntary participation. CLABS unifies global unlinkability and fine-grained, context-specific linkage within a single formal framework.

We realize CLABS generically using three modular components: a pseudorandom function for deterministic tag generation, a conventional signature for attribute certification, and a signature of knowledge (SoK) proving correct tag computation and Boolean policy satisfaction without revealing x. Finally, we instantiate CLABS under standard lattice assumptions in the quantum random oracle model (QROM), achieving post-quantum security while supporting arbitrary Boolean policies. The techniques we employ to prove circuit satisfiability and tag correctness may be of independent interest.

⋆ This work is supported under Research and Development Agreement between Technology Innovation Institute (TII) and the University of Wollongong. W. Susilo is also supported by the Australian Laureate Fellowship (FL230100033).

1 Introduction

Attribute-Based Signatures. Introduced by Maji, Prabhakaran, and Rosulek [32], attribute-based signature (ABS) is a digital signature primitive enabling fine-grained authentication with signer privacy. In an ABS scheme, a user possessing an attribute x certified by an authority can anonymously sign a message M under a signing policy C, provided that $C(x) = 1$. The resulting signature proves that *someone* whose certified attribute satisfies the policy, produced the message, without revealing any additional information about that user's identity or attribute. Thanks to its dual control over authorization and anonymity, ABS has attracted extensive study for its potential applications in attribute-based messaging, authentication and trust negotiation, leakage-resilient reporting, and non-transferable access control (see [32,27] for early discussions).

Research themes in ABS. Since its introduction, the development of ABS has evolved along three main research axes.

(i) Expressiveness of signing policies. A major line of work seeks to broaden the expressiveness of admissible signing policies. Okamoto and Takashima [34] constructed efficient ABS schemes for non-monotone access structures, extending the initial monotone-only model of [32]. Subsequent works supported increasingly general policy classes such as bounded-depth circuits [43], unbounded arithmetic branching programs [12], and non-deterministic finite automata [39]. This culminated in constructions realizing policies expressed as arbitrary circuits [38,17] and even Turing machines [39]. These developments place expressive ABS on par with its encryption analogue (ABE), establishing a universal abstraction for predicate-based authentication.

(ii) Diversity of computational assumptions. A second thread focuses on instantiating ABS under diverse hardness assumptions. Following the pairing-based schemes of Maji et al. [32], numerous efficient realizations appeared within bilinear settings [18,16]. The first non-pairing scheme was proposed by Herranz [25], while the advent of post-quantum cryptography spurred lattice-based and code-based designs (e.g., [3,17,22,30]). These advances show that ABS, like ABE, can be instantiated from standard assumptions in both classical and quantum-resilient settings.

(iii) Functional extensions and new features. A third, and arguably most conceptually fruitful, direction has been the quest to enrich ABS with new functionalities beyond policy expressiveness. Examples include decentralized ABS [35] (eliminating the single authority), traceable and accountable ABS [18,16] (adding group-signature-like opening mechanisms), ABS with limited forms of linkability [15,45], forward-secure ABS [48], hierarchical ABS [14], ABS with revocability [28,6,40,22,30], and generalizations regarding the control of "who can sign messages" such as policy-based signatures [4,11] and multimodal private signatures [33,41].

Despite this extensive body of work, all existing extensions share a structural rigidity: they treat anonymity and linkability as global, static properties. A signature is either *always unlinkable* (as in standard ABS), or *universally linkable*

across all contexts (as in linkable or accountable ABS variants). Even the notions of *controllable linkability* [15] and *user-controlled linkability* [45] fail to overcome this rigidity. The former delegates linking authority to an external entity that can globally decide when two signatures should be correlated—reintroducing central trust; the latter allows a signer to voluntarily reuse a local pseudonym, but lacks any policy semantics or public verifiability. Neither supports a structured, attribute-conditioned mechanism where linkability arises only under mutually declared, context-specific rules.

Motivating conditional linkability. In many applications, absolute unlinkability and global linkability represent two extremes, neither of which captures nuanced privacy requirements. Consider an anonymous reporting platform where participants contribute data about different projects. Rather than tracking the continuity of a single anonymous user, a verifier may need to publicly correlate submissions from *different signers* whose certified attributes satisfy a context-dependent rule (e.g., belonging to the same organization or regulatory class), while keeping all other interactions unlinkable. Traditional ABS offers no mechanism to express such cross-user, attribute-driven correlation. If signatures are always unlinkable, correlation is impossible; if linkability is global, privacy is lost. This tension highlights a fundamental missing capability in the ABS landscape.

Importantly, this goal is fundamentally different from pseudonym-based approaches (e.g., [45,31]). Reusable pseudonyms merely allow the same user to be recognized across sessions, whereas CLABS enables publicly verifiable linkage across multiple users based solely on certified attributes and declared contexts.

Modern distributed systems increasingly require cryptographic mechanisms that enable *context-dependent correlation*: signers remain anonymous in general, but their actions may become linkable under explicitly declared contexts. We refer to this missing capability as *conditional linkability*.

Our Contributions. Our goal is to bridge the gap between the global privacy of classical ABS and the practical need for context-specific correlation. This work introduces the first formal model and realization of *Conditionally Linkable Attribute-Based Signatures* (CLABS)— a new class of privacy-preserving signatures that extend the traditional ABS framework with context-dependent linkability. Beyond this conceptual advance, our results also contribute along two additional ABS research axes: (i) *Expressiveness:* our construction supports arbitrary Boolean signing circuits through an explicit NAND-based zero-knowledge encoding while retaining public verifiability; and (ii) *Assumption diversity:* our concrete instantiation achieves post-quantum security from standard assumptions over general lattices, without relying on structured variants, contrasting with previous pairing-based or module-lattice ABS schemes. Together, these advances establish CLABS as a feature-rich, modular, and assumption-generic generalization of attribute-based signatures. In addition to the formal model, we present a generic construction and a QROM-secure lattice-based instantiation, showing that conditional linkability can be realized concretely within standard post-quantum frameworks.

In a CLABS system, each certified user with attribute $x \in \mathcal{A}$ is associated with a set of *linking contexts* $L_x \subseteq \mathcal{T}$, where $\mathcal{T}$ is a public space of admissible signing contexts (e.g., projects, audit rounds, regulatory domains, or time epochs). Each context $\tau \in \mathcal{T}$ also specifies a public function $f_\tau : \mathcal{A} \to \{0,1\}^\ell$ that determines how attributes are compared under that context.

Two valid signatures Σ and Σ' are publicly linkable at τ if and only if

$$\tau \in L_x \cap L_{x'} \quad \text{and} \quad f_\tau(x) = f_\tau(x').$$

Otherwise, they remain unlinkable, revealing no information about the underlying attributes beyond what is required for policy satisfaction. Intuitively, linkability arises only when *both signers have opted into* the same linking context and their attributes are related under the publicly defined function f_τ. The only observable fact in an unlinkable case is that at least one signer's linking set does not contain τ—a minimal and unavoidable form of policy visibility. This design provides a clean and principled separation between attribute privacy and policy visibility: the public can learn which contexts are auditable, but not the attributes or identities of the signers themselves.

By elevating linkability from a global property to a programmable, context-dependent relation, CLABS captures a spectrum of anonymity–linkability trade-offs within a single unified model. Ordinary ABS corresponds to the special case where $L_x = \emptyset$ for all users (complete unlinkability), and a fully linkable variant arises when $L_x = \mathcal{T}$. Between these two extremes lies a wide range of practically motivated configurations in which linkability is conditional, attribute-driven, and publicly verifiable. Such fine-grained control is not only theoretically elegant but also crucial for designing privacy-preserving infrastructures that remain *operationally functional*—supporting coordination and aggregation without compromising anonymity.

We next illustrate these advantages through several representative application domains that naturally align with the CLABS syntax and semantics.

1. Attribute-class accountability. Consider signers whose certified attributes have the form $x = (\texttt{role}, \texttt{organization}, \texttt{region})$. Let an auditing context τ_{audit} be fixed, and define $f_{\tau_{\mathsf{audit}}}(x) = \texttt{organization}$. Each participant that agrees to organizational accountability includes $\tau_{\mathsf{audit}} \in L_x$. Then any two signatures whose hidden attributes share the same organization become linkable in this context, enabling public verifiers to check consistency of statements within each organization. If a signature is not linkable, observers merely learn that its signer has not opted into the audit context ($\tau_{\mathsf{audit}} \notin L_x$); nothing else about its attribute is revealed.

2. Service continuity and scoped pseudonyms. Suppose x encodes a researcher's identity and project memberships, and τ designates a particular project session. Here, linkability does not arise from pseudonym reuse: it is determined solely by the public relation f_τ on certified attributes and does not reveal or rely on persistent user identities. Define $f_\tau(x) = \texttt{project\_id}(x)$. If a participant includes $\tau \in L_x$, then all of their submissions under that project can be publicly recognized as belonging to the same project group, facilitating updates and tracking.

If another participant with the same attribute omits τ from L_x, their submissions remain unlinkable and the only visible implication is that they have not joined the linkable session. This minimal leakage is intentional—it allows systems to distinguish between signers who consented to contextual correlation and those who did not, without revealing which attribute values they hold.

3. Statistical soundness and deduplication. In anonymous data collection, each signer's attribute may include $(\texttt{sector}, \texttt{region})$. For a survey round τ, define $f_\tau(x) = \texttt{region}$ and require $\tau \in L_x$ for contributors who consent to per-region deduplication. All signatures with the same region value under τ link together, ensuring one report per region. Unlinked signatures either correspond to other regions or to users who declined participation in this survey ($\tau \notin L_x$)—information that must be public for verifiability but does not disclose which region or sector a non-participant belongs to.

4. Policy-aware compliance and selective transparency. Each entity may hold an attribute $x = (\texttt{entity\_type}, \texttt{license\_class}, \texttt{id})$. For a compliance context τ_{reg}, the authority defines $f_{\tau_{\mathsf{reg}}}(x) = \texttt{license\_class}(x)$. Entities under regulatory oversight have $\tau_{\mathsf{reg}} \in L_x$, so their signatures can be publicly correlated by license class for audits. Others, such as private-sector participants, exclude τ_{reg} from their linking set. Hence, an unlinked signature in this context simply reveals non-participation in the regulatory domain ($\tau_{\mathsf{reg}} \notin L_x$), preserving confidentiality beyond that single policy bit.

5. Functional inter-class linkage. Because f_τ can encode arbitrary deterministic relations, it supports controlled correlation beyond attribute equality. For example, if $f_\tau(x) = \texttt{industry\_code}(x)$ outputs the industry code derived from an organization's certified classification, then all entities operating in the same industry can be linked under τ, supporting public transparency at the sector level. An unlinkable signature indicates only that the signer did not opt into this sectoral context or belongs to a distinct sector class— precisely the limited, policy-intended information that CLABS exposes.

Across all these settings, the public learns *only* whether $\tau \in L_x$ for a given signature and, if so, the equivalence class of $f_\tau(x)$ shared by all linked signatures. This minimal, structural leakage is what makes CLABS functionally useful: it allows verifiers to perform consistency checks, and aggregate data at the policy granularity defined by f_τ, while maintaining complete anonymity and unlinkability elsewhere.

Syntax and Definitions of CLABS. A CLABS scheme extends the syntax of standard Attribute-Based Signatures (ABS) by introducing a *linking set* for each user and a *context parameter* for signing and verification.

Formally, a CLABS system is defined over parameters $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$, where $\mathcal{A}$ is the attribute space, $\mathcal{C}$ a family of Boolean circuits $C : \mathcal{A} \to \{0,1\}$, $\mathcal{T}$ a public set of signing contexts, and $\mathcal{F} = \{f_\tau : \mathcal{A} \to \{0,1\}^\ell\}_{\tau \in \mathcal{T}}$ a set of publicly known linking functions. Each signer with attribute x declares a linking set $L_x \subseteq \mathcal{T}$, $|L_x| = k$, indicating the contexts in which her signatures may be linkable.

A CLABS scheme consists of algorithms

$$(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify}, \mathsf{Link}),$$

sharing the same interface as ABS, except for the following extensions:

- **Key generation.** $\mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x, L_x)$ outputs a signing key sk_{x,L_x} bound to both the user's attribute x and the declared linking set L_x.
- **Signing and verification.** $\mathsf{Sign}(sk_{x,L_x}, M, C, \tau)$ and $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau)$ both take a context $\tau \in \mathcal{T}$ as input, determining whether linkability applies.
- **Linking.** $\mathsf{Link}(\mathsf{pp}, \Sigma, \Sigma', \tau)$ outputs 1 if and only if

$$\tau \in L_x \cap L_{x'} \quad \text{and} \quad f_\tau(x) = f_\tau(x'),$$

  meaning that the two signatures are linkable precisely when both signers have opted into the same context and their certified attributes coincide under f_τ.

If $\tau \notin L_x$, the signature remains fully unlinkable and reveals no additional information about (x, L_x) beyond policy satisfaction.

While the correctness and unforgeability properties of CLABS follow naturally from those of ABS, the core technical subtleties lie in two intertwined properties: *linkability* and *anonymity*. Each extends traditional attribute-based notions into a conditional setting where privacy and verifiable correlation coexist under explicit contextual control.

Linkability. Unlike the global linkability of group or linkable signatures, CLABS realizes *conditional linkability*: two signatures become publicly linkable only when both users have voluntarily opted into the same context τ and their certified attributes satisfy the public relation $f_\tau(x) = f_\tau(x')$. This context-driven correlation must be publicly verifiable yet sound, requiring an *extraction-based* definition: from any valid linkable transcript, an extractor must recover a consistent attribute–certificate pair confirming genuine participation. This guarantee prevents adversaries from producing artificial linkable collisions or generating "pseudo-identities" without legitimate attributes. Thus, linkability in CLABS captures structural consistency across contexts, not personal identifiability.

Anonymity. Privacy in CLABS bifurcates into two complementary forms. When $\tau \notin L_x$, signatures achieve *unlinkable anonymity*: they are fully simulatable and reveal nothing beyond policy satisfaction, mirroring the classical ABS notion of privacy. When $\tau \in L_x$, the scheme provides *linkable anonymity*: signatures may be publicly linkable through the tag $f_\tau(x)$, yet this tag is pseudorandom and hides all attribute information except equality under f_τ. This formulation enables a coherent coexistence of anonymity and context-dependent correlation within a single security model.

Together, these refinements make CLABS a non-trivial generalization of ABS: it replaces static, global privacy with a programmable, extractable, and provably sound balance between anonymity and verifiable contextual linkage.

Constructions of CLABS. Having established the syntax and security goals, we now describe how CLABS can be realized. Our design proceeds in two layers:

a generic, assumption-agnostic construction that abstracts the cryptographic structure of conditional linkability, and a concrete lattice-based instantiation that achieves post-quantum security.

Generic construction. At a high level, our generic CLABS framework integrates three canonical primitives: a pseudorandom function (PRF), a conventional signature scheme (for attribute certification), and a signature of knowledge (SoK) system proving the correctness of the signing process. The PRF deterministically enforces context-dependent tags $t = (\tau, f_\tau(x))$ that enable linkability, the signature scheme guarantees certified attribute ownership, and the SoK ensures that each signature is well-formed without revealing the underlying attribute. This modular composition provides a clear separation between policy enforcement (via the circuit C and certified attribute x), contextual correlation (through the pair (τ, f_τ)), and anonymity (through the zero-knowledge layer).

A signature in this framework has the form $\Sigma = (M, C, \tau, t, \pi)$, where t equals the deterministic tag $(\tau, f_\tau(x))$ (if $\tau \in L_x$) or a fixed dummy value 0^ℓ otherwise, and π is a SoK attesting to the consistency of all components. Correctness and linkability follow from algebraic consistency of t and the extractability of the SoK, while anonymity relies on its simulatability and the pseudorandomness of the PRF. These reductions isolate each security property under a distinct primitive assumption, yielding a clean, composable proof structure that can be instantiated under a variety of cryptographic paradigms.

Conceptually, this abstraction elevates CLABS beyond a single realization: it serves as a universal template for constructing privacy-preserving, policy-aware signature systems that combine certified attributes, verifiable computation, and controlled linkability. The framework thus provides a blueprint for exploring conditional linkability under alternative assumptions—such as code-, isogeny-, or MPC-based foundations— while preserving the same modular reasoning and security structure.

Lattice-based instantiation. We instantiate this framework under standard lattice assumptions in the Quantum Random Oracle Model (QROM). Our construction combines three ingredients operating over general (i.e., unstructured) lattices: the LWR-based pseudorandom function in [19], the SIS-based signature with efficient protocols by Jeudy et al. [26] (JRS) for attribute certification, and a zero-knowledge layer derived from the lattice Σ-protocol of Yang et al. [47], converted into a SoK via the Unruh transform [44]. The resulting CLABS realization achieves post-quantum security and supports arbitrary Boolean signing circuits. It remains fully modular, allowing future instantiations to substitute alternative post-quantum primitives without altering the high-level logic.

The JRS component authenticates certified attributes, the LWR-based PRF deterministically generates context-specific tags, and the zero-knowledge (SoK) layer ties them together by proving that each signature is correctly formed without revealing the underlying attribute x or its linking status. While these parts align with the generic framework, the technically most intricate element is the *SoK layer*, which must simultaneously encode circuit satisfiability $C(x) = 1$ and

enforce correct handling of the hidden membership bit b indicating whether the current context τ belongs to L_x.

Well-formedness of the membership bit and circuit handling. The SoK proves knowledge of (x, L_x, b) satisfying

$$C(x) = 1, \qquad b \in \{0,1\}, \qquad \mathbf{t} = b \cdot \mathbf{y}_{x,\tau},$$

where $\mathbf{y}_{x,\tau}$ is the token tied to $\tau \in L_x$. The constraint $b(b-1) = 0$ enforces that b is a valid bit. A one-hot selector vector $\mathbf{u} = (u_1, \ldots, u_k)$ with $u_i \in \{0,1\}$ and $\sum_i u_i = b$ binds b to a concrete entry in L_x. If $b = 0$, all $u_i = 0$ and $\mathbf{t} = \mathbf{0}$ (unlinkable case); if $b = 1$, exactly one $u_j = 1$ with $\tau = \tau_j$, giving $\mathbf{t} = \mathbf{y}_{x,\tau}$ (linkable case).

To enforce the policy condition $C(x) = 1$ for an arbitrary Boolean circuit, we express C in NAND form, since every Boolean gate can be rewritten as $z = \mathrm{NAND}(x_1, x_2) = 1 - x_1 x_2$. Each gate of C is translated into an arithmetic constraint

$$z + x_1 x_2 = 1,$$

and the SoK proves that all such relations hold over the committed wires. This enables circuit satisfiability to be verified entirely through additive and multiplicative checks, compatible with the lattice-based Σ-protocol of [47]. Together, the circuit equations and membership constraints form a unified relation over short integer vectors, linking policy compliance ($C(x) = 1$) with linkability correctness $(b, \mathbf{t})$ within one extractable proof system.

Zero-knowledge and extraction. We apply multiple repetitions of the Σ-protocol from [47] to achieve negligible soundness error, then transform it non-interactively using Unruh's technique [44]. The resulting SoK satisfies: (i) simulatability under the Learning With Errors (LWE) assumption, ensuring anonymity; and (ii) extractability under the Short Integer Solution (SIS) assumption, ensuring soundness and verifiable linkability. From any accepting transcript, the extractor recovers a valid witness (x, L_x, b), thereby determining whether $\tau \in L_x$ and whether the tag $\mathbf{t}$ corresponds to a genuine PRF output or the dummy value $\mathbf{0}$.

This mechanism provides the core technical guarantee of our lattice-based instantiation: conditional linkability is proven within a unified relation endowed with a knowledge-sound proof system, where the hidden bit b serves as the cryptographic switch between unlinkable and linkable modes. Its well-formedness ensures that adversaries cannot generate "partially linkable" or inconsistent signatures, yielding a clean, extractable boundary between the unlinkable and linkable domains.

We conclude by highlighting an intriguing open direction. While our lattice-based instantiation achieves post-quantum security in the QROM, obtaining a CLABS construction secure *in the standard model* remains an important challenge. Recent advances in lattice-based proof systems and signatures of knowledge (e.g., [36,46,29,42]) suggest that such a development may be within reach. Bridging conditional linkability with standard-model lattice techniques would

further strengthen the conceptual foundations of CLABS and expand its applicability within post-quantum cryptography.

Organization. The rest of the paper is organized as follows. In Section 2, we formalize the syntax and define the security notions for CLABS. Our generic construction is presented in Section 3 and its analyses are given in Section 4. Section 5 then provides our lattice-based instantiation of the generic construction. Due to the lack of space, the reminders on commonly used cryptographic building blocks and details of the zero-knowledge protocol supporting the lattice-based CLABS scheme are deferred to Appendix A and Appendix B, respectively.

2 Conditionally Linkable Attribute-Based Signatures

2.1 Syntax

Let $\lambda \in \mathbb{N}$ be the security parameter. Any Conditionally Linkable Attribute-Based Signature (CLABS) system is associated with two positive integers $k, \ell \in \mathsf{poly}(\lambda)$; a message space $\mathcal{M} \subseteq \{0,1\}^\star$; an attribute space $\mathcal{A} \subseteq \{0,1\}^*$; a collection of polynomial-size Boolean circuits $\mathcal{C} : \mathcal{A} \to \{0,1\}$; a set of admissible signing contexts $\mathcal{T} \subseteq \{0,1\}^*$ and a collection of functions $\mathcal{F} = \{f_\tau : \mathcal{A} \to \{0,1\}^\ell \mid \tau \in \mathcal{T}\}$. We assume that each $\tau \in \mathcal{T}$ uniquely determines a function $f_\tau \in \mathcal{F}$ and the mapping $\tau \mapsto f_\tau$ is publicly known and specified as part of the system's public parameters.

A CLABS scheme is managed by an authority that is in charge of setting up and enrolling users into the system. To register, a potential user with attribute $x \in \mathcal{A}$ interacts with the authority to first determine a set of "linking contexts" $L_x \subseteq \mathcal{T}$ with respect to its attribute x, where $|L_x| = k$, and then receive a signing key sk_{x,L_x}. The latter allows the user to generate a valid signature Σ on any message $M \in \mathcal{M}$ with respect to any signing context $\tau \in \mathcal{T}$, and any circuit $C \in \mathcal{C}$ such that $C(x) = 1$. The privacy and linkability of Σ depends on whether the signing context τ is subject to linking, i.e., whether $\tau \in L_x$, as follows:

- If $\tau \in L_x$, then Σ can be publicly linked with any other valid signature Σ' associated with the same context τ, under the conditions that Σ' was generated by a registered user with attribute x', whose corresponding set $L_{x'}$ also contains τ. In that case, the two signatures are linked if and only if $f_\tau(x) = f_\tau(x')$.
- Otherwise, i.e., when $\tau \in \mathcal{T} \setminus L_x$, then Σ is unlinkable and it leaks no additional information about (x, L_x), even against a corrupted authority.

Formally, a CLABS scheme associated with $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ is a tuple of polynomial-time algorithms $(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify}, \mathsf{Link})$, defined as follows.

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{pp}, \mathsf{msk})$: On input the security parameter λ, this algorithm generates public parameters pp and a master secret key msk. We assume that the public parameters include the descriptions of $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$.

- $\mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x, L_x) \to sk_{x,L_x}/\perp$: On input the public parameters pp, the master secret key msk, an attribute $x \in \mathcal{A}$ and a set $L_x \subseteq \mathcal{T}$ s.t. $|L_x| = k$ consisting of linking contexts for attribute x, this algorithm outputs a secret key sk_{x,L_x}.
- $\mathsf{Sign}(sk_{x,L_x}, M, C, \tau) \to \Sigma$: On input a secret key sk_{x,L_x}, a message $M \in \mathcal{M}$, a circuit $C \in \mathcal{C}$ and a context $\tau \in \mathcal{T}$, this algorithm outputs a signature Σ.
- $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) \to 0/1$: On input the public parameters pp, a message $M \in \mathcal{M}$, a circuit $C \in \mathcal{C}$, a signature Σ, and a context $\tau \in \mathcal{T}$, this algorithm outputs a bit $v \in \{0,1\}$, indicating whether the signature is valid or not.
- $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) \to 0/1$: On input the public parameters pp, two valid signatures Σ_0, Σ_1 and a context $\tau \in \mathcal{T}$, this algorithm outputs a bit $b \in \{0,1\}$, indicating whether the two signatures are linkable or not.

2.2 Correctness and Security Notions

A CLABS scheme should satisfy the correctness and security properties defined in the following.

Intuitively, correctness requires that honestly generated signatures always verify, that unlinkable contexts remain unlinkable, and that signatures corresponding to the same link condition are consistently linked.

Definition 1 (Correctness). *We say that a CLABS scheme specified by the parameter tuple* $(k,\ \ell,\ \mathcal{M},\ \mathcal{A},\ \mathcal{C},\ \mathcal{T},\ \mathcal{F})$ *is* correct *if the following hold for all security parameters* $\lambda \in \mathbb{N}$.

(1) Verification correctness. *For every attribute* $x \in \mathcal{A}$*, policy* $C \in \mathcal{C}$ *with* $C(x)=1$*, context set* $L_x \subseteq \mathcal{T}$ *of size* k*, and message* $M \in \mathcal{M}$*, let*

$$(\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda),\ \ sk_{x,L_x} \leftarrow \mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x, L_x),\ \ \Sigma \leftarrow \mathsf{Sign}(sk_{x,L_x}, M, C, \tau).$$

Then the verification succeeds with overwhelming probability:

$$\Pr\left[\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) = 1\right] \geq 1 - \mathsf{negl}(\lambda),$$

where the probability is taken over the internal randomness of Setup*,* KeyGen*, and* Sign*.*

(2) Linking correctness. *For any honest users with attributes* $x_0, x_1 \in \mathcal{A}$*, context sets* $L_{x_0}, L_{x_1} \subseteq \mathcal{T}$ *of size* k*, messages* $M_0, M_1 \in \mathcal{M}$*, and valid circuits* $C_0, C_1 \in \mathcal{C}$ *satisfying* $C_i(x_i)=1$ *for* $i \in \{0,1\}$*, let*

$$(\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda), \quad sk_{x_i,L_{x_i}} \leftarrow \mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x_i, L_{x_i}),$$

and $\Sigma_i \leftarrow \mathsf{Sign}(sk_{x_i,L_{x_i}}, M_i, C_i, \tau)$*, for some* $\tau \in \mathcal{T}$*. Then, with probability at least* $1 - \mathsf{negl}(\lambda)$ *over all randomness:*

- *If* $\tau \notin L_{x_i}$ *for some* $i \in \{0,1\}$*, then* $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = 0$.
- *If* $\tau \in L_{x_0} \cap L_{x_1}$ *and* $f_\tau(x_0) = f_\tau(x_1)$*, then* $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = 1$.

Before defining the security properties of CLABS, we first specify three databases used throughout the attack games.

- **Key-generation log** $\mathcal{L}_{\mathsf{KeyGen}}$: Initially empty. Each entry has the form (x, L_x, sk_{x,L_x}), recording the attribute x and the corresponding secret key returned in a key-generation query.
- **Signing log** $\mathcal{L}_{\mathsf{Sign}}$: Initially empty. Each entry has the form (M, x, C, τ, Σ), indicating that signature Σ was returned in a signing query involving attribute x, message M, circuit C, and context τ.
- **Corruption log** $\mathcal{L}_{\mathsf{Corrupt}}$: Initially empty. Each entry of this database has the form (x, L_x, sk_{x,L_x}), representing attributes whose secret keys are known to the adversary.

During the games, the adversary may interact with the following oracles.

- $O_{\mathsf{KeyGen}}(x, L_x)$**:** On input an attribute $x \in \mathcal{A}$ and a list of contexts L_x, if $(x, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGen}}$ or $|L_x| \neq k$, the oracle does nothing. Otherwise, it computes

$$sk_{x,L_x} \leftarrow \mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x, L_x),$$

  updates $\mathcal{L}_{\mathsf{KeyGen}} \leftarrow \mathcal{L}_{\mathsf{KeyGen}} \cup \{(x, L_x, sk_{x,L_x})\}$, and returns nothing.
- $O_{\mathsf{Corrupt}}(x)$**:** If $(x, L_x, \cdot) \in \mathcal{L}_{\mathsf{KeyGen}}$ for some L_x, return the key tuple $(sk_{x,\tau})_{\tau \in L_x}$ and update $\mathcal{L}_{\mathsf{Corrupt}} \leftarrow \mathcal{L}_{\mathsf{Corrupt}} \cup \{(x, L_x, sk_{x,L_x})\}$. Otherwise, return $\perp$.
- $O_{\mathsf{Sign}}(M, x, C, \tau)$**:** On input an attribute $x \in \mathcal{A}$, message $M \in \mathcal{M}$, circuit $C \in \mathcal{C}$, and context $\tau \in \mathcal{T}$:
  - Return $\perp$ if $C(x) = 0$, or if $(x, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KeyGen}} \setminus \mathcal{L}_{\mathsf{Corrupt}}$.
  - Otherwise, retrieve (x, L_x, sk_{x,L_x}) from $\mathcal{L}_{\mathsf{KeyGen}} \setminus \mathcal{L}_{\mathsf{Corrupt}}$, compute

$$\Sigma \leftarrow \mathsf{Sign}(sk_{x,L_x}, M, C, \tau),$$

    update $\mathcal{L}_{\mathsf{Sign}} := \mathcal{L}_{\mathsf{Sign}} \cup \{(M, x, C, \tau, \Sigma)\}$, and return Σ.

Auxiliary Algorithms, Setup Indistinguishability, and Linkability. The syntax of CLABS alone does not allow one to check whether the output of the Link algorithm behaves correctly in adversarially generated scenarios. To formalize this property, we introduce two auxiliary algorithms that enable extraction under a simulated setup.

- $\mathsf{SimSetup}(1^\lambda) \to (\mathsf{pp}, \mathsf{msk}, \mathsf{tr})$: On input the security parameter λ, this algorithm outputs a simulated public parameter pp, a simulated master secret key msk, and a trapdoor tr. We require that the public parameters produced by $\mathsf{SimSetup}$ and Setup are computationally indistinguishable.
- $\mathsf{Extract}(\mathsf{pp}, \mathsf{tr}, M, C, \Sigma, \tau) \to (x, L_x)$: On input the public parameter pp, trapdoor tr, message $M \in \mathcal{M}$, circuit $C \in \mathcal{C}$, and context $\tau \in \mathcal{T}$, together with a valid signature Σ satisfying $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) = 1$, this algorithm deterministically extracts a tuple (x, L_x).

These algorithms allow the challenger to recover the attribute x and its context set L_x from adversarially produced signatures. Intuitively, the $\mathsf{Extract}$ oracle enables the experiment to test whether the Link algorithm behaves as intended: given two signatures (Σ_0, Σ_1) over context τ with extracted pairs (x_0, L_{x_0}) and (x_1, L_{x_1}), Link should output

$$\begin{cases} 0, & \text{if } \tau \notin L_{x_i} \text{ for some } i \in \{0,1\}, \\ 1, & \text{if } \tau \in L_{x_0} \cap L_{x_1} \text{ and } f_\tau(x_0) = f_\tau(x_1), \\ 0, & \text{otherwise.} \end{cases}$$

In the real setup, the challenger has no way to recover (x_i, L_{x_i}) directly from the signatures, hence the need for $(\mathsf{SimSetup}, \mathsf{Extract})$. Moreover, to prevent the adversary from distinguishing between the real and simulated environments, we require that the transcripts generated by Setup and $\mathsf{SimSetup}$ (excluding the trapdoor) be computationally indistinguishable. This is formalized below. Note that in Definition 3, it is sufficient to let $\mathcal{A}$ have access to O_{KeyGen} and O_{Corrupt} as such oracles allow $\mathcal{A}$ to create any signatures to make the linking experiment fair to $\mathcal{A}$, since we ask $\mathcal{A}$ to provide signatures that pass verification before extracting and checking Link.

Definition 2 (Setup Indistinguishability). *A CLABS scheme associated with parameters* $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ *satisfies* setup indistinguishability *if, for every PPT adversary* $\mathcal{A}$*,*

$$\Big| \Pr\big[\mathcal{A}(\mathsf{pp}, \mathsf{msk}) = 1 \;\big|\; (\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda)\big] - \Pr\big[\mathcal{A}(\mathsf{pp}, \mathsf{msk}) = 1 \;\big|\; (\mathsf{pp}, \mathsf{msk}, \mathsf{tr}) \leftarrow \mathsf{SimSetup}(1^\lambda)\big] \Big| \leq \mathsf{negl}(\lambda).$$

Definition 3 (Linkability). *We say that a CLABS scheme associated with* $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ *satisfies* linkability *if, for every PPT adversary* $\mathcal{A}$*, its advantage*

$$\mathsf{Adv}^{\mathsf{Link}}_{\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{Link}}_{\mathcal{A}}(\lambda) = 1]$$

is negligible in λ*, where the experiment* $\mathbf{Exp}^{\mathsf{Link}}_{\mathcal{A}}(\lambda)$ *is defined in Figure 1.*

Unforgeability. For unforgeability, we follow the standard definition for attribute-based signature schemes [32,38]. Intuitively, an adversary should not be able to forge a valid tuple (τ, M, C, Σ) that either (i) corresponds to a query previously answered by the signing oracle O_{Sign}, or (ii) is valid for some attribute x whose secret key has been revealed through O_{Corrupt}. If either case were allowed, $\mathcal{A}$ could trivially produce signatures that always verify successfully. Since CLABS signatures are parameterized by a context τ, we require that (τ, M, C, Σ) itself—not merely (M, C)— has not been queried before; the inclusion of Σ also captures *strong unforgeability.*

Definition 4 (Unforgeability). *A CLABS scheme associated with parameters* $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ *is* unforgeable *if, for every PPT adversary* $\mathcal{A}$*, its advantage*

$$\mathsf{Adv}^{\mathsf{UF}}_{\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{UF}}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

$(\mathsf{pp}, \mathsf{msk}, \mathsf{tr}) \leftarrow \mathsf{SimSetup}(1^\lambda)$.
$(\tau, M_0, M_1, C_0, C_1, \Sigma_0, \Sigma_1) \leftarrow \mathcal{A}^{O_{\mathsf{KeyGen}}, O_{\mathsf{Corrupt}}}(\mathsf{pp})$.
if $\exists\ i\ \in \{0,1\},\ \mathsf{Verify}(\mathsf{pp}, M_i, C_i, \Sigma_i, \tau) = 0$ **then**
  return 0.
$(x_i, L_{x_i}) \leftarrow \mathsf{Extract}(\mathsf{pp}, \mathsf{tr}, M_i, C_i, \Sigma_i, \tau)\ \forall\ i\ \in\ \{0,1\}$.
if $\tau \in L_{x_0} \wedge \tau \in L_{x_1}$ **then**
  if $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = 1 \wedge f_\tau(x_0) \neq f_\tau(x_1)$ **then**
    return 1.
  if $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = 0 \wedge f_\tau(x_0) = f_\tau(x_1)$ **then**
    return 1.
else
  if $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = 1$ **then**
    return 1.
return 0.

Fig. 1: Experiment $\mathbf{Exp}^{\mathsf{Link}}_{\mathcal{A}}(\lambda)$

$(\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda)$;
$(\tau, M, C, \Sigma) \leftarrow \mathcal{A}^{O_{\mathsf{KeyGen}}, O_{\mathsf{Corrupt}}, O_{\mathsf{Sign}}}(\mathsf{pp})$;
if $\exists\, x:\ (M, x, C, \tau, \Sigma) \in \mathcal{L}_{\mathsf{Sign}}$ **then**
  return 0;
if $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) = 0$ **then**
  return 0;
if $\exists\, (x, \cdot, \cdot) \in \mathcal{L}_{\mathsf{Corrupt}}$ *with* $C(x) = 1$ **then**
  return 0;
return 1.

Fig. 2: Experiment $\mathbf{Exp}^{\mathsf{UF}}_{\mathcal{A}}(\lambda)$

where the experiment $\mathbf{Exp}^{\mathsf{UF}}_{\mathcal{A}}(\lambda)$ *is described in Figure 2.*

Anonymity. Unlike classical constructions, our CLABS cannot achieve perfect anonymity, since an adversary can use the Link algorithm to test whether a challenge signature is linkable to previously obtained ones. Accordingly, we distinguish two cases depending on the linkability of the challenge signature.

- *Unlinkable anonymity.* When a signature is *unlinkable* (i.e., $\tau \notin L_x$), it should reveal nothing about the signer's attribute, even if the adversary knows the master secret key. In the challenge phase, $\mathcal{A}$ outputs $(M, C, x_0, x_1, \tau, \mathsf{st})$ such that $\tau \notin L_{x_i}$ for all $i \in \{0,1\}$, and receives $\Sigma_b \leftarrow \mathsf{Sign}(sk_{x_b}, M, C, \tau)$. Its goal is to guess b. We call this **unlinkable anonymity**.
- *Linkable anonymity.* When a signature is *linkable* (i.e., $\tau \in L_x$), the adversary may exploit linkability but should learn nothing beyond whether $f_\tau(x)$

equals $f_\tau(x')$ for attributes x' that the adversary can get a signature from x' previously (if $\tau \in L_{x'}$). In this case, the adversary does not obtain msk. We call this **linkable anonymity**.

Definition 5 (Unlinkable Anonymity). *We say that a CLABS associated with $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ satisfies unlinkable anonymity if for any PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, it holds that*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{UL\text{-}Anon}}(\lambda) = \left| \Pr[\mathbf{Exp}_{\mathcal{A},0}^{\mathsf{UL\text{-}Anon}}(\lambda) = 1] - \Pr[\mathbf{Exp}_{\mathcal{A},1}^{\mathsf{UL\text{-}Anon}}(\lambda) = 1] \right| \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathbf{Exp}_{\mathcal{A},b}^{\mathsf{UL\text{-}Anon}}(\lambda)$ is described in Figure 3.

$(\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda)$.
$(M, C, x_0, x_1, \tau, st) \leftarrow \mathcal{A}_0^{O_{\mathsf{KeyGen}}, O_{\mathsf{Corrupt}}, O_{\mathsf{Sign}}}(\mathsf{pp}, \mathsf{msk})$.
if $\exists\ b \in \{0,1\}$ *s.t. no* $(L_{x_b}, sk_{x_b, L_{x_b}})$ *satisfies* $(x_b, L_{x_b}, sk_{x_b, L_{x_b}}) \in \mathcal{L}_{\mathsf{KeyGen}}$ **then**
  return 0.
if $\exists\ b \in \{0,1\}$ *s.t.* $\tau \in L_{x_b}$ **then**
  return 0.
$\Sigma_b \leftarrow \mathsf{Sign}(sk_{x_b}, M, C, \tau)$.
$b' \leftarrow \mathcal{A}_1(\mathsf{pp}, \mathsf{msk}, M, C, x_0, x_1, \tau, \Sigma_b, st)$.
return b'.

Fig. 3: Experiment $\mathbf{Exp}_{\mathcal{A},b}^{\mathsf{UL\text{-}Anon}}(\lambda)$

Definition 6 (Linkable Anonymity). *We say that a CLABS associated with $(k, \ell, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{T}, \mathcal{F})$ satisfies linkable anonymity if for any PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, it holds that*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{L\text{-}Anon}}(\lambda) = \left| \Pr[\mathbf{Exp}_{\mathcal{A},0}^{\mathsf{L\text{-}Anon}}(\lambda) = 1] - \Pr[\mathbf{Exp}_{\mathcal{A},1}^{\mathsf{L\text{-}Anon}}(\lambda) = 1] \right| \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathbf{Exp}_{\mathcal{A},b}^{\mathsf{L\text{-}Anon}}(\lambda)$ is described in Figure 4.

Discussion. The sets $D_{\tau,0}$ and $D_{\tau,1}$ ensure that no previously queried attribute x allows the adversary to distinguish the challenge signatures via the Link oracle. If $\tau \in L_x$ for some queried x, the adversary could form a signature Σ and compute $\mathsf{Link}(\mathsf{pp}, \Sigma, \Sigma_i, \tau)$ to determine whether $f_\tau(x) = f_\tau(x_i)$, thus trivially revealing b. By restricting to instances where either $f_\tau(x) = f_\tau(x_i)$ for all i or $f_\tau(x) \neq f_\tau(x_i)$ for all i, the adversary learns no additional information. Hence, this definition captures the strongest achievable anonymity for linkable signatures.

$(\mathsf{pp}, \mathsf{msk}) \leftarrow \mathsf{Setup}(1^\lambda)$.

$(M, C, x_0, x_1, \tau, st) \leftarrow \mathcal{A}_0^{O_{\mathsf{KeyGen}}, O_{\mathsf{Corrupt}}, O_{\mathsf{Sign}}}(\mathsf{pp})$.

if $\exists\ b \in \{0,1\}$ *s.t. no* $(L_{x_b}, sk_{x_b, L_{x_b}})$ *satisfies* $(x_b, L_{x_b}, sk_{x_b, L_{x_b}}) \in \mathcal{L}_{\mathsf{KeyGen}}$ **then**
  return 0.

if $\exists\ b \in \{0,1\}$ *s.t.* $\tau \notin L_{x_b}$ **then**
  return 0.

Let

$$D_{\tau,0} = \{x \in \mathcal{A} \mid x, \cdot, \cdot, \tau, \cdot) \in \mathcal{L}_{\mathsf{Sign}} \ \wedge\ \tau \in L_x\},$$
$$D_{\tau,1} = \{x \in \mathcal{A} \mid (x, \cdot, \cdot) \in \mathcal{L}_{\mathsf{Corrupt}} \ \wedge\ \tau \in L_x\}.$$

if $\exists\ x \in D_{\tau,0} \cup D_{\tau,1}$ *s.t.* $(f_\tau(x) = f_\tau(x_0) \wedge f_\tau(x) \neq f_\tau(x_1)) \vee (f_\tau(x) \neq f_\tau(x_0) \wedge f_\tau(x) = f_\tau(x_1))$ **then**
  return 0.

$\Sigma_b \leftarrow \mathsf{Sign}(sk_{x_b}, M, C, \tau)$.

$b' \leftarrow \mathcal{A}_1(\mathsf{pp}, M, C, x_0, x_1, \tau, \Sigma_b, st)$.

return b'.

Fig. 4: Experiment $\mathbf{Exp}^{\mathsf{L\text{-}Anon}}_{\mathcal{A},b}(\lambda)$

3 A Generic Construction of CLABS

Our CLABS construction combines three standard cryptographic components. First, for each context τ, the public map $f_\tau : \mathcal{A} \to \{0,1\}^\ell$ projects an attribute x to a linking value $v_{x,\tau} = f_\tau(x)$. Second, the authority, holding a PRF key K, derives per-context tokens $y_{x,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, v_{x,\tau})$ for all $\tau \in L_x$, and certifies the tuple $(x, \{(\tau, y_{x,\tau})\}_{\tau \in L_x})$ using its signing key. Third, to sign (M, C, τ), the user publishes a tag t equal to the certified token $y_{x,\tau}$ if $\tau \in L_x$, and $t = 0^\ell$ otherwise, together with a Signature of Knowledge (SoK) proving that (i) the authority's certificate is valid, (ii) the policy circuit C accepts the attribute ($C(x)=1$), and (iii) t is consistent with (x, L_x) and the certified tokens. Verification thus reduces to a single SoK check, while linking consists of equality of nonzero tags.

NP relation used by the SoK. Let vk_A be the authority's verification key (from SIG). We define a relation R over statements (C, τ, t) and witnesses $w = (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A)$ such that $((C, \tau, t); w) \in R$ iff

(i) $\mathsf{SIG.Verify}(\mathsf{vk}_A, (x, \{(\tau', y_{x,\tau'})\}_{\tau' \in L_x}), \sigma_A) = 1$;
(ii) $C(x) = 1$;
(iii) $t = b \cdot y_{x,\tau}$, where $b = 1$ if $\tau \in L_x$ and $b = 0$ otherwise;

Intuitively, R captures certified ownership of (x, L_x) and tokens, policy satisfaction, and correct tag formation.

Specifically, we employ a PRF $\mathsf{PRF} = (\mathsf{PRF.Setup}, \mathsf{PRF.KeyGen}, \mathsf{PRF.Eval})$, a signature scheme $\mathsf{SIG} = (\mathsf{SIG.Setup}, \mathsf{SIG.KeyGen}, \mathsf{SIG.Sign}, \mathsf{SIG.Verify})$, and a

signature-of-knowledge system $\mathsf{SoK} = (\mathsf{SoK.Setup}, \mathsf{SoK.Sign}, \mathsf{SoK.Verify})$ instantiated for relation R. Our CLABS construction works as follows.

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{pp}, \mathsf{msk})$**:**
  - Initialize the PRF: $\mathsf{pp}_{\mathsf{PRF}} \leftarrow \mathsf{PRF.Setup}(1^\lambda)$ and sample $K \leftarrow \mathsf{PRF.KeyGen}(\mathsf{pp}_{\mathsf{PRF}})$ (this fixes the output length ℓ).
  - Generate authority signing keys: first obtain $\mathsf{pp}_{\mathsf{SIG}} \leftarrow \mathsf{SIG.Setup}(1^\lambda)$, then generate $(\mathsf{vk}_A, \mathsf{sk}_A) \leftarrow \mathsf{SIG.KeyGen}(\mathsf{pp}_{\mathsf{SIG}})$.
  - Instantiate the SoK system: $\mathsf{pp}_{\mathsf{SoK}} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)$.
  - Publish
    $$\mathsf{pp} = (\lambda, k, \ell, \mathsf{pp}_{\mathsf{PRF}}, \mathsf{pp}_{\mathsf{SIG}}, \mathsf{pp}_{\mathsf{SoK}}, \mathsf{vk}_A, \mathcal{C}, \mathcal{T}, \mathcal{F}),$$
    where $\mathcal{F}$ is the family $\{f_\tau\}_{\tau \in \mathcal{T}}$; keep $\mathsf{msk} = (K, \mathsf{sk}_A)$ secret.
- $\mathsf{KeyGen}(\mathsf{pp}, \mathsf{msk}, x, L_x) \to sk_{x,L_x}$**:**
  - For each $\tau \in L_x$, compute $v_{x,\tau} = (\tau, f_\tau(x))$ and $y_{x,\tau} \leftarrow \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, v_{x,\tau})$.
  - Encode $m_x = (x, \{(\tau, y_{x,\tau}) \mid \tau \in L_x\})$ and compute the authority's certificate $\sigma_A \leftarrow \mathsf{SIG.Sign}(\mathsf{sk}_A, m_x)$.
  - Output $sk_{x,L_x} = (x, L_x, \{y_{x,\tau}\}_{\tau \in L_x}, \sigma_A)$.
- $\mathsf{Sign}(sk_{x,L_x}, M, C, \tau) \to \Sigma$**:**
  - Parse $sk_{x,L_x} = (x, L_x, \{y_{x,\tau'}\}, \sigma_A)$ and set
    $$t = \begin{cases} y_{x,\tau}, & \text{if } \tau \in L_x, \\ 0^\ell, & \text{otherwise.} \end{cases}$$
  - Using witness $w = (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A)$ to compute
    $$\pi \leftarrow \mathsf{SoK.Sign}(\mathsf{pp}_{\mathsf{SoK}}, R, M, (C, \tau, t), w).$$
  - Output $\Sigma = (M, C, \tau, t, \pi)$.
- $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) \to \{0,1\}$**:** Parse $\Sigma = (M, C, \tau, t, \pi)$ and accept if and only if $\mathsf{SoK.Verify}(\mathsf{pp}_{\mathsf{SoK}}, R, M, (C, \tau, t), \pi) = 1$.
- $\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) \to \{0,1\}$**:** Parse $\Sigma_i = (M_i, C_i, \tau, t_i, \pi_i)$ for $i \in \{0,1\}$, ensure both verify, and output 1 iff $t_0 = t_1$ and $t_0, t_1 \neq 0^\ell$; otherwise output 0.

Auxiliary algorithms. We also instantiate the auxiliary algorithms $\mathsf{SimSetup}$ and $\mathsf{Extract}$ (see Section 2.2):

- $\mathsf{SimSetup}(1^\lambda)$: identical to Setup, except that $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda)$; output $(\mathsf{pp}, \mathsf{msk}, \mathsf{tr})$.
- $\mathsf{Extract}(\mathsf{pp}, \mathsf{tr}, M, C, \Sigma, \tau)$: if $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) = 0$, abort; otherwise parse (M, C, τ, t, π) from Σ and compute
  $$(x, \cdot, L_x, \cdot) \leftarrow \mathsf{SoK.Extract}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, R, M, (C, \tau, t))$$
  and return (x, L_x).

Roughly speaking, correctness follows because equal $f_\tau(x)$ values yield equal PRF tokens and hence identical nonzero tags whenever both signers include $\tau \in L_x$; otherwise, at least one tag is 0^ℓ or the tokens differ, so no link is produced. Anonymity and policy privacy follow from the zero-knowledge property of the SoK and the pseudorandomness of the PRF: beyond validity and the visible tag t, the signature reveals nothing about x or L_x, except the unavoidable information that linked signatures share a context $\tau \in L_x$ or that at least one signer omitted τ for unlinkable contexts. We provide the formal analyses in Section 4.

4 Analyses of the Generic Construction

In this section, we prove the security of the generic construction presented in Section 3. Throughout, let R denote the relation defined therein. Also, for simplicity, in the queries O_{KeyGen}, we assume that $\mathcal{A}$ always query (x, L_x) s.t. $|L_x| = k$, because otherwise, the challenger or any reduction adversary $\mathcal{A}'$ can ignore such queries.

4.1 Correctness

We first show that our CLABS construction satisfies the correctness property.

Theorem 1. *Assume:*

(i) the underlying SIG *and* SoK *are correct; and*
(ii) for any input X, a value $y = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, X) = 0^\ell$ occurs only with negligible probability over $(\mathsf{pp}_{\mathsf{PRF}}, K) \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.

Then the construction in Section 3 satisfies correctness (Definition 1).

Proof. Correctness follows directly from the definitions. By the correctness of SoK and the structure of the relation R, we have that the tag t—computed as $t = y_{x,\tau}$ if $\tau \in L_x$ and $t = 0^\ell$ otherwise—can equivalently be expressed as $t = b \cdot y_{x,\tau}$, where $b = 1$ if $\tau \in L_x$ and $b = 0$ otherwise. Hence

$$((C, \tau, t), (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A)) \in R,$$

and since Σ is generated honestly using SoK.Sign on this instance–witness pair, verification accepts with probability $1 - \mathsf{negl}(\lambda)$.

For the linking algorithm, if $\tau \notin L_{x_i}$ for some i, then $t_i = 0^\ell$ and thus Link outputs 0. Otherwise, $t_i = y_{x_i,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_i)))$ for each $i \in \{0, 1\}$. By assumption, $y_{x_i,\tau} = 0^\ell$ only with negligible probability. If $f_\tau(x_0) = f_\tau(x_1)$, then $y_{x_0,\tau} = y_{x_1,\tau}$, so Link returns 1 with overwhelming probability, as required. □

4.2 Setup Indistinguishability

We now show that our CLABS construction satisfies the setup indistinguishability property.

Theorem 2. *Assuming the parameter indistinguishability of* SoK*, the construction in Section 3 satisfies setup indistinguishability (Definition 3).*

Proof. The proof is immediate. The only difference between Setup and $\mathsf{SimSetup}$ lies in the initialization of the SoK component:

$$\mathsf{Setup} : \mathsf{pp}_{\mathsf{SoK}} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R), \;\; \mathsf{SimSetup} : (\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R).$$

Since the distributions of $\mathsf{pp}_{\mathsf{SoK}}$ in both procedures are computationally indistinguishable by the parameter indistinguishability of SoK, it follows that the public transcripts produced by Setup and $\mathsf{SimSetup}$ are themselves indistinguishable, as required. □

4.3 Linkability

We now prove that our CLABS construction satisfies the linkability property.

Theorem 3. *Assuming (i)* SIG *satisfies unforgeability, (ii)* SoK *satisfies extractability, and (iii)* PRF *is collision resistant, the construction in Section 3 satisfies linkability (Definition 3).*

Proof. Suppose $\mathcal{A}$ outputs $(\tau, M_0, M_1, C_0, C_1, \Sigma_0, \Sigma_1)$, where each $\Sigma_i = (y_i, \sigma_i)$. Let

$$W_i = (x_i, L_{x_i}, \{y_{x_i,\tau'}\}_{\tau' \in L_{x_i}}, \sigma_{A,i})$$

be the witness extracted by

$$\mathsf{SoK.Extract}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M_i, (C_i, \tau, t_i), \sigma_i).$$

Thus (x_i, L_{x_i}) is also the value obtained from $\mathsf{Extract}$. Let $X_i = (C_i, \tau, t_i)$ for brevity.

When $\mathcal{A}$ wins the experiment, define:

- $P_{1,0}$: $(X_0, W_0) \notin R$,
- $P_{1,1}$: $(X_1, W_1) \notin R$,
- P_2: $(X_i, W_i) \in R$ for both i.

It is obvious that one of the events $P_{1,0}, P_{1,1}$ and P_2 must occur. Therefore we consider each event. If $P_{1,0}$ occurs with non-negligible probability, it is straightforward to see that one can construct an adversary $\mathcal{B}$ such that $\Pr[P_{1,0}] \leq \mathsf{Adv}_{\mathcal{B}}^{\mathsf{SoK\text{-}Ext}}(\lambda)$, breaking the extractability of SoK (for more details see the adversary $\mathcal{A}'$ in the proof of Section 4.4. The adversary $\mathcal{B}$ here is the same as the adversary $\mathcal{A}'$ in in the proof of Section 4.4). The same holds for $P_{1,1}$. Hence it suffices to show that $\Pr[P_2]$ is negligible.

If P_2 occurs, then $(X_i, W_i) \in R$ for $i \in \{0, 1\}$. We analyze two cases.

Case 1. $\tau \notin L_{x_0}$ or $\tau \notin L_{x_1}$. Without loss of generality, $\tau \notin L_{x_0}$. Then $t_0 = 0^\ell$, hence Link always returns 0, contradicting the win condition. Thus this case cannot happen.

Case 2. $\tau \in L_{x_0} \cap L_{x_1}$. Two subcases arise:

(a) Incorrect PRF evaluation. If $y_{x_i,\tau} \neq \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_i)))$ for some i, define $P_{3,i}$ for this event. Then there exists an adversary $\mathcal{A}'$ with $\Pr[P_{3,i}] \leq \mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}'}(\lambda)$ for all i (see Lemma 1); hence each $\Pr[P_{3,i}]$ is negligible.

(b) Correct PRF evaluation. Then $\forall i$: $y_{x_i,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_i)))$. For $\mathcal{A}$ to win, one of the following must hold:

1. $f_\tau(x_0) = f_\tau(x_1)$ but Link returns 0. Since $f_\tau(x_0) = f_\tau(x_1)$, we have $t_0 = t_1$ and thus Link must output 1, contradiction.
2. $f_\tau(x_0) \neq f_\tau(x_1)$ but Link returns 1. Let P_4 denote this event. Since $(X_i, W_i) \in R$, each t_i equals $\mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_i)))$. If P_4 occurs, we obtain a collision for PRF, contradicting its collision resistance (Lemma 2).

Summing all cases yields

$$\begin{aligned}\mathsf{Adv}^{\mathsf{Link}}_{\mathcal{A}}(\lambda) &\leq \Pr[P_{1,0}] + \Pr[P_{1,1}] + \Pr[P_2] \\ &\leq 2\mathsf{Adv}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{B}}(\lambda) + 2\mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}'}(\lambda) + \mathsf{Adv}^{\mathsf{PRF\text{-}Col}}_{\mathcal{A}''}(\lambda),\end{aligned}$$

so $\mathcal{A}$ wins with only negligible probability. □

Lemma 1. *There exists an adversary $\mathcal{A}'$ such that $\Pr[P_{3,i}] \leq \mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}'}(\lambda)$ for all $i \in \{0, 1\}$.*

Proof. Without loss of generality, take $i = 0$. Adversary $\mathcal{A}'$ receives $\mathsf{pk}_{\mathsf{SIG}}$ from its challenger, generates $(\mathsf{pp}_{\mathsf{PRF}}, K)$ and $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr})$, and provides the corresponding setup parameters to $\mathcal{A}$.

For oracle queries:

- On $O_{\mathsf{KeyGen}}(x, L_x)$, store (x, L_x) without action.
- On $O_{\mathsf{Corrupt}}(x)$, if $(x, \cdot)$ not stored, return $\perp$. Otherwise, compute $y_{x,\tau'} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, f_\tau(x))$ for each $\tau' \in L_x$, encode $m_x = \big(x, \{(\tau, y_{x,\tau})\}_{\tau \in L_x}\big)$, query m_x to its challenger, and obtain σ_A. Return $sk_{x,L_x} = (x, L_x, \{y_{x,\tau}\}, \sigma_A)$.

Finally, $\mathcal{A}$ outputs $(\tau, M_0, M_1, C_0, C_1, \Sigma_0, \Sigma_1)$. Adversary $\mathcal{A}'$ extracts $W_0 = (x_0, L_{x_0}, \{y_{x_0,\tau'}\}_{\tau' \in L_{x_0}}, \sigma_{A,0})$ and submits $(m_{x_0}, \sigma_{A,0})$ to its challenger. Note that m_{x_0} is not a message queried by $\mathcal{A}'$ in this case because for every message m_x queried, the component $(\tau, y_{x,\tau})$ must satisfy $y_{x,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K)$. Since $P_{3,0}$ implies $\mathsf{SIG.Verify}(\mathsf{pk}_{\mathsf{SIG}}, m_{x_0}, \sigma_{A,0}) = 1$ and m_{x_0} was not previously queried, this constitutes a valid forgery. Hence $\Pr[P_{3,0}] \leq \mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}'}(\lambda)$, as claimed. □

Lemma 2. *There exists an adversary $\mathcal{A}''$ such that $\Pr[P_4] \leq \mathsf{Adv}^{\mathsf{PRF\text{-}Col}}_{\mathcal{A}''}(\lambda)$.*

Proof. Adversary $\mathcal{A}''$ receives $(\mathsf{pp}_{\mathsf{PRF}}, K)$ from its challenger, sets up all other parameters honestly, and provides them to $\mathcal{A}$. It answers O_{KeyGen} and O_{Corrupt} honestly. When $\mathcal{A}$ outputs $(\tau, M_0, M_1, C_0, C_1, \Sigma_0, \Sigma_1)$, $\mathcal{A}''$ extracts $y_{x_0,\tau}$ and $y_{x_1,\tau}$ and submits $(\tau, f_\tau(x_0))$ and $(\tau, f_\tau(x_1))$ to its PRF challenger. If P_4 occurs, then $t_0 = t_1$ but $f_\tau(x_0) \neq f_\tau(x_1)$, implying a collision for PRF. Thus $\Pr[P_4] \leq \mathsf{Adv}^{\mathsf{PRF\text{-}Col}}_{\mathcal{A}''}(\lambda)$. □

4.4 Unforgeability

We now show that our CLABS construction satisfies the unforgeability property.

Theorem 4. *Assuming (i)* SIG *satisfies unforgeability and (ii)* SoK *satisfies both simulatability and extractability, the construction in Section 3 satisfies unforgeability (Definition 3).*

Proof. We analyze a sequence of hybrid games.

Game 0. This is the original unforgeability game.

Game 1. Same as Game **0**, except that during setup we run $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda)$, and in Sign we generate σ using $\mathsf{SoK.SimSign}$ instead of $\mathsf{SoK.Sign}$.

By the simulatability of SoK (Definition 13), Games **0** and **1** are computationally indistinguishable. Otherwise, one could construct an adversary $\mathcal{B}$ breaking simulatability as follows. $\mathcal{B}$ receives $\mathsf{pp}_{\mathsf{SoK}}$ from its challenger, then generates the remaining parameters $(\mathsf{pp}_{\mathsf{PRF}}, \mathsf{pp}_{\mathsf{SIG}})$ and $\mathsf{msk} = (K, \mathsf{sk}_A)$, and forwards pp to $\mathcal{A}$. It answers O_{KeyGen}, O_{Corrupt}, and O_{Sign} honestly, except that for signing queries it forwards the message-instance–witness pair

$$(M, (C, \tau, t), (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A))$$

to its SoK challenger and returns the resulting proof π as part of $\Sigma = (M, C, \tau, t, \pi)$.

Finally, when $\mathcal{A}$ outputs (τ, M, C, Σ), $\mathcal{B}$ outputs 1 iff: (i) $(\cdot, M, C, \tau, \Sigma) \notin \mathcal{L}_{\mathsf{Sign}}$, (ii) $\mathsf{Verify}(\mathsf{pp}, M, C, \Sigma, \tau) = 1$, and (iii) no $(x, \cdot) \in \mathcal{L}_{\mathsf{Corrupt}}$ satisfies $C(x) = 1$. When $\mathsf{pp}_{\mathsf{SoK}}$ and π originate from the simulator, $\mathcal{A}$ plays Game **1**; otherwise, it plays Game **0**. Thus,

$$\left|\Pr[\mathsf{Game}_0 = 1] - \Pr[\mathsf{Game}_1 = 1]\right| \leq \mathsf{Adv}^{\mathsf{SoK\text{-}Sim}}_{\mathcal{B}}(\lambda),$$

which is negligible.

It therefore suffices to prove that $\Pr[\mathsf{Game}_1 = 1]$ is negligible. Suppose otherwise. $\mathcal{A}$ outputs (τ, M, C, Σ) where $\Sigma = (M, C, \tau, t, \pi)$. Let

$$W = (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A) \leftarrow \mathsf{SoK.Extract}(\mathsf{pp}_{\mathsf{SoK}}, R, M, (C, \tau, t)), \;\; X = (C, \tau, t).$$

Define:

- P_1: $\mathcal{A}$ wins and $(X, W) \notin R$;
- P_2: $\mathcal{A}$ wins and $(X, W) \in R$.

We show that both $\Pr[P_1]$ and $\Pr[P_2]$ are negligible.

Bounding $\Pr[P_1]$. There exists an adversary $\mathcal{A}'$ such that $\Pr[P_1] \leq \mathsf{Adv}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{A}'}(\lambda)$ (Definition 12). $\mathcal{A}'$ proceeds as follows:

- Receives $\mathsf{pp}_{\mathsf{SoK}}$ from its challenger; generates $(\mathsf{pp}_{\mathsf{PRF}}, K)$ and $(\mathsf{pk}_{\mathsf{SIG}}, \mathsf{sk}_{\mathsf{SIG}})$; sends $\mathsf{pp} = (\mathsf{pp}_{\mathsf{PRF}}, \mathsf{pk}_{\mathsf{SIG}}, \mathsf{pp}_{\mathsf{SoK}})$ to $\mathcal{A}$.
- On $O_{\mathsf{KeyGen}}(x, L_x)$, if $(x, L_x, \cdot) \notin \mathcal{L}_{\mathsf{KeyGen}}$, compute

$$y_{x,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x))),$$

  encode $m_x = (x, \{(\tau, y_{x,\tau})\}_{\tau \in L_x})$, sign $\sigma_A \leftarrow \mathsf{SIG.Sign}(\mathsf{sk}_{\mathsf{SIG}}, m_x)$, and store (x, L_x, sk_{x,L_x}).
- On $O_{\mathsf{Corrupt}}(x)$, return the stored sk_{x,L_x} if any, otherwise $\perp$.
- On $O_{\mathsf{Sign}}(M, x, C, \tau)$, if $C(x) = 1$ and $(x, \cdot) \in \mathcal{L}_{\mathsf{KeyGen}} \setminus \mathcal{L}_{\mathsf{Corrupt}}$, parse sk_{x,L_x}, compute t depending on $\tau \in L_x$, forward $(M, (C, \tau, t), (x, L_x, \{y_{x,\tau'}\}, \sigma_A))$ to its challenger to receive π, then return $\Sigma = (M, C, \tau, t, \pi)$.

Finally, when $\mathcal{A}$ outputs $(M, C, \tau, \Sigma = (M, C, \tau, t, \pi))$, then $\mathcal{A}'$ parses π from Σ and forwards the tuple $(M, (C, \tau, t), \pi)$ to its challenger. Note that, because $(\cdot, M, C, \tau, \Sigma)$ was never stored in $\mathcal{L}_{\mathsf{Sign}}$, the corresponding message-instance-signature list $(M, (C, \tau, t), \pi)$ was stored in the signing query of the SoK challenger when interacting with $\mathcal{A}'$. (indeed, due to the way $\mathcal{A}'$ handling O_{Sign}, a list $(M, (C, \tau, t), \pi)$ is stored only if $\mathcal{A}$ has forwarded $(\cdot, M, C, \tau, t)$, then $\mathcal{A}'$ forwards $(M, (C, \tau, t), \cdot)$ and received $\Sigma = (M, C, \tau, t, \pi)$). As a result, if (C, τ, t) is invalid (since $(X, W) \notin R$), then $\mathcal{A}'$ breaks extractability of SoK, hence $\Pr[P_1] \leq \mathsf{Adv}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{A}'}(\lambda)$.

Bounding $\Pr[P_2]$. If $\Pr[P_2]$ is non-negligible, there exists $\mathcal{A}''$ with $\Pr[P_2] \leq \mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}''}(\lambda)$. Adversary $\mathcal{A}''$ operates as follows:

- Receive $\mathsf{pk}_{\mathsf{SIG}}$ from its challenger and initialize $\mathcal{L}_{\mathsf{Corrupt}}$. Generate $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda)$ and $(\mathsf{pp}_{\mathsf{PRF}}, K) \leftarrow \mathsf{PRF.Setup}(1^\lambda)$, returning pp to $\mathcal{A}$.
- On $O_{\mathsf{KeyGen}}(x, L_x)$, store (x, L_x) without action.
- On $O_{\mathsf{Corrupt}}(x)$, if $(x, \cdot)$ not stored, return $\perp$. Otherwise compute $y_{x,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x)))$, encode m_x and query it to the challenger to obtain σ_A, then return $sk_{x,L_x} = (x, L_x, \{y_{x,\tau}\}, \sigma_A)$ and record it in $\mathcal{L}_{\mathsf{Corrupt}}$.
- On $O_{\mathsf{Sign}}(M, x, C, \tau)$, if $(x, \cdot)$ was generated but not corrupted, compute t as before and obtain $\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, R, \mathsf{tr}, M, (C, \tau, t))$, then return $\Sigma = (M, C, \tau, t, \pi)$.

Finally, when $\mathcal{A}$ outputs (τ, M, C, Σ), $\mathcal{A}''$ extracts $W = (x, L_x, \{y_{x,\tau'}\}_{\tau' \in L_x}, \sigma_A)$, computes $m_x = (x, \{(\tau, y_{x,\tau})\}_{\tau \in L_x})$, and submits (m_x, σ_A) to its challenger. Since W is valid, we have $\mathsf{SIG.Verify}(\mathsf{pk}_{\mathsf{SIG}}, m_x, \sigma_A) = 1$ and $C(x) = 1$. Moreover, m_x was never queried to the challenger (because corruption queries require $C(x) = 0$), so (m_x, σ_A) is a valid forgery. Thus $\Pr[P_2] \leq \mathsf{Adv}^{\mathsf{SIG\text{-}UF}}_{\mathcal{A}''}(\lambda)$.

Putting all together. By the union bound,

$$\Pr[\mathsf{Game}_0 = 1] \leq \Pr[\mathsf{Game}_1 = 1] + \mathsf{Adv}_{\mathcal{B}}^{\mathsf{SoK\text{-}Sim}}(\lambda) + \Pr[P_1] + \Pr[P_2],$$

and hence

$$\Pr[\mathsf{Game}_0 = 1] \leq \mathsf{Adv}_{\mathcal{B}}^{\mathsf{SoK\text{-}Sim}}(\lambda) + \mathsf{Adv}_{\mathcal{A}'}^{\mathsf{SoK\text{-}Ext}}(\lambda) + \mathsf{Adv}_{\mathcal{A}''}^{\mathsf{SIG\text{-}UF}}(\lambda).$$

Therefore, the scheme achieves unforgeability. □

4.5 Unlinkable Anonymity

In this section, we prove that our CLABS construction satisfies unlinkable anonymity.

Theorem 5. *Assuming that* SoK *satisfies the simulatability property, the construction in Section 3 achieves unlinkable anonymity (Definition 5).*

Proof. We proceed via a standard hybrid argument.

Game $\mathbf{0}_b$. This is the original unlinkable anonymity experiment $\mathbf{Exp}_{\mathcal{A},b}^{\mathsf{UL\text{-}Anon}}(\lambda)$, as defined in Figure 3.

Game $\mathbf{1}_b$. Same as Game $\mathbf{0}_b$, except that:

- In Setup, instead of running $\mathsf{pp}_{\mathsf{SoK}} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)$, we execute the simulator's setup $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R)$.
- In Sign, instead of running $\pi \leftarrow \mathsf{SoK.Sign}(\mathsf{pp}_{\mathsf{SoK}}, R, M, (C, \tau, t), w)$, we execute the simulated signing algorithm $\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, t))$.

Indistinguishability of Game $\mathbf{0}_b$ and Game $\mathbf{1}_b$. By the simulatability of SoK (Definition 13), the two games are computationally indistinguishable for each fixed bit $b \in \{0, 1\}$. Formally, there exists a negligible function $\mathsf{negl}(\lambda)$ such that

$$\left| \Pr[\mathbf{Game\ 0_b} = 1] - \Pr[\mathbf{Game\ 1_b} = 1] \right| \leq \mathsf{negl}(\lambda).$$

Indistinguishability of Game $\mathbf{1}_0$ and Game $\mathbf{1}_1$. It remains to show that the distributions of Game $\mathbf{1}_0$ and Game $\mathbf{1}_1$ are identical, even when the adversary is given the master secret key msk.

The only difference between the two games is which secret key is used to produce the challenge signature: $\Sigma_b \leftarrow \mathsf{Sign}(sk_{x_b}, M, C, \tau)$. However, by the experiment's requirement, $\tau \notin L_{x_i}$ for all $i \in \{0, 1\}$. Therefore, in both games, the signing algorithm sets $t = 0^\ell$ and invokes the simulated signature of knowledge:

$$\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, t)).$$

Hence, in both games, the challenge signature is identically distributed as

$$\Sigma_b = (M, C, \tau, 0^\ell, \pi),$$

and thus the full game transcripts are perfectly indistinguishable. That is,

$$\Pr[\mathbf{Game\ 1_0} = 1] = \Pr[\mathbf{Game\ 1_1} = 1].$$

Putting everything together, we have:

$$\begin{aligned}\big|\Pr[\mathbf{Game\ 0_0} = 1] - \Pr[\mathbf{Game\ 0_1} = 1]\big| \leq\ & \big|\Pr[\mathbf{Game\ 0_0} = 1] - \Pr[\mathbf{Game\ 1_0} = 1]\big| \\ & + \big|\Pr[\mathbf{Game\ 0_1} = 1] - \Pr[\mathbf{Game\ 1_1} = 1]\big| \\ & + \big|\Pr[\mathbf{Game\ 1_0} = 1] - \Pr[\mathbf{Game\ 1_1} = 1]\big|.\end{aligned}$$

The first two terms are negligible by simulatability of SoK, and the last term is zero due to perfect equality of the distributions. Therefore,

$$\big|\Pr[\mathbf{Game\ 0_0} = 1] - \Pr[\mathbf{Game\ 0_1} = 1]\big| \leq \mathsf{negl}(\lambda),$$

implying unlinkable anonymity of the scheme. □

4.6 Linkable Anonymity

Compared to the unlinkable anonymity proof, which relies solely on the simulatability of the SoK system and holds trivially when $\tau \notin L_x$ (since all signatures are simulated with $t = 0^\ell$), the proof of *linkable anonymity* is substantially more intricate. Here, the challenge tag τ may lie within the linking domain of both challenge users, and thus the tag value t in the challenge signature encodes an actual pseudorandom output $y_{x,\tau} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x)))$. Consequently, we can no longer rely purely on SoK simulatability: we must also ensure that this value is indistinguishable from a uniformly random one, even when $\mathcal{A}$ has obtained many other PRF evaluations on related inputs. Therefore, the reduction combines the SoK simulatability property with the *weak pseudorandomness* of the PRF to ensure that the adversary cannot link two otherwise anonymous signatures that share the same tag τ. This interdependence between SoK and PRF security makes the proof more sophisticated than that of unlinkable anonymity.

Theorem 6. *Assuming that (i)* SoK *satisfies the simulatability property and (ii)* PRF *satisfies the weak pseudorandomness property, the construction in Section 3 achieves linkable anonymity (Definition 6).*

Proof. We consider a sequence of hybrid games as follows.

Game $\mathbf{0}_b$. This is the original linkable anonymity experiment $\mathbf{Exp}^{\mathsf{L\text{-}Anon}}_{\mathcal{A},b}(\lambda)$, as defined in Figure 4.

Game $\mathbf{1}_b$. Same as Game $\mathbf{0}_b$, except that:

- In Setup, instead of running $\mathsf{pp}_{\mathsf{SoK}} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)$, we execute the simulator's setup $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R)$.
- In Sign, instead of running $\pi \leftarrow \mathsf{SoK.Sign}(\mathsf{pp}_{\mathsf{SoK}}, R, M, (C, \tau, t), w)$, we execute the simulated signing algorithm $\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, t))$.

Indistinguishability of Game $\mathbf{0}_b$ and Game $\mathbf{1}_b$. By the simulatability of SoK (Definition 13), these two games are computationally indistinguishable for every fixed $b \in \{0,1\}$:

$$\left|\Pr[\textbf{Game } \mathbf{0_b} = 1] - \Pr[\textbf{Game } \mathbf{1_b} = 1]\right| \leq \mathsf{negl}(\lambda).$$

It thus suffices to prove that **Game $\mathbf{1_0}$** and **Game $\mathbf{1_1}$** are indistinguishable.

Reduction to PRF pseudorandomness. Suppose there exists an adversary $\mathcal{A}$ that distinguishes between **Game $\mathbf{1_0}$** and **Game $\mathbf{1_1}$**. We construct an adversary $\mathcal{A}'$ that breaks the weak pseudorandomness of the PRF (Definition 16) as follows.

- $\mathcal{A}'$ receives $\mathsf{pp}_{\mathsf{PRF}}$ from the PRF challenger. It then samples $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda)$ and $(\mathsf{pk}_{\mathsf{Sig}}, \mathsf{sk}_{\mathsf{Sig}}) \leftarrow \mathsf{Sig.KeyGen}(1^\lambda)$, and returns the combined public parameter pp to $\mathcal{A}$.
- On $O_{\mathsf{KeyGen}}(x, L_x)$: $\mathcal{A}'$ maintains a local list $\mathcal{L}'_{\mathsf{KeyGen}}$. Upon a query, it records (x, L_x) in $\mathcal{L}'_{\mathsf{KeyGen}}$ and does nothing else.
- On $O_{\mathsf{Corrupt}}(x)$: $\mathcal{A}'$ maintains lists $\mathcal{L}'_{\mathsf{PRF}}$, $\mathcal{L}_{\mathsf{Corrupt}}$, and D. When $\mathcal{A}$ corrupts user x, $\mathcal{A}'$ checks $(x, L_x) \in \mathcal{L}'_{\mathsf{KeyGen}}$, queries each $(\tau, f_\tau(x))$ to the PRF challenger for $\tau \in L_x$, and receives $y_{x,\tau}$. It updates

  $$\mathcal{L}'_{\mathsf{PRF}} := \mathcal{L}'_{\mathsf{PRF}} \cup \{((\tau, f_\tau(x)), y_{x,\tau})\}_{\tau \in L_x}, \quad D := D \cup \{(x, \tau, y_{x,\tau})_{\tau \in L_x}\}.$$

  It then computes $m_x = (x, \{(\tau, y_{x,\tau})\}_{\tau \in L_x})$ and $\sigma_A \leftarrow \mathsf{Sig.Sign}(\mathsf{sk}_{\mathsf{Sig}}, m_x)$, returns $sk_{x,L_x} = (x, L_x, \{y_{x,\tau}\}, \sigma_A)$ to $\mathcal{A}$, and records it in $\mathcal{L}_{\mathsf{Corrupt}}$.
- On $O_{\mathsf{Sign}}(M, x, C, \tau)$: $\mathcal{A}'$ maintains $\mathcal{L}'_{\mathsf{PRF}}$, $\mathcal{L}_{\mathsf{Sign}}$, and D. If $C(x) \neq 1$ or $(x, L_x) \notin \mathcal{L}'_{\mathsf{KeyGen}}$ or $(x, \cdot) \in \mathcal{L}_{\mathsf{Corrupt}}$, it returns $\perp$. Otherwise:
  1. If $(x, \tau, y_{x,\tau}) \in D$, then $t = y_{x,\tau}$ (since $\tau \in L_x$ and the value is known).
  2. Else, if $\tau \in L_x$, query $(\tau, f_\tau(x))$ to the PRF challenger to obtain $y_{x,\tau}$, update D and $\mathcal{L}'_{\mathsf{PRF}}$, and set $t = y_{x,\tau}$; otherwise set $t = 0^\ell$.

  Then run $\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, t))$ and output $\Sigma = (M, C, \tau, t, \pi)$ to $\mathcal{A}$, updating $\mathcal{L}_{\mathsf{Sign}}$ accordingly.
- At challenge time, $\mathcal{A}$ outputs $(M, C, x_0, x_1, \tau, \cdot)$. $\mathcal{A}'$ proceeds as follows:
  1. If $(x_i, \cdot) \notin \mathcal{L}'_{\mathsf{KeyGen}}$ for some i, it queries dummy inputs to the PRF challenger and to receive $y^\star$ returns 0.
  2. If $\tau \notin L_{x_b}$ for some b, it also returns 0.
  3. Otherwise, if $\tau \in L_{x_i}$ for both i:
     - *Case 1 (event P):* For all $x \in D_{\tau,0} \cup D_{\tau,1}$, $f_\tau(x) \neq f_\tau(x_i)$ for all i. By Lemma 3, $(\tau, f_\tau(x_i))$ has not been queried to the PRF challenger. Thus, $\mathcal{A}'$ submits the two challenge inputs $X_0 = (\tau, f_\tau(x_0))$ and $X_1 = (\tau, f_\tau(x_1))$ to the PRF challenger and receives a value $y^\star$. It then computes $\pi^\star \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, y^\star))$ and returns $\Sigma^\star = (M, C, \tau, y^\star, \pi^\star)$ to $\mathcal{A}$.
     - *Case 2:* There exists $x \in D_{\tau,0} \cup D_{\tau,1}$ such that $f_\tau(x) = f_\tau(x_0) = f_\tau(x_1)$. Then $((\tau, f_\tau(x)), y) \in \mathcal{L}'_{\mathsf{PRF}}$ for some known y. In this case, $\mathcal{A}'$ simply runs $\pi \leftarrow \mathsf{SoK.SimSign}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, M, (C, \tau, y))$ and returns (y, π) to $\mathcal{A}$ then queries dummy challenge inputs to the PRF challenger to receive $y^\star$.

Finally, $\mathcal{A}'$ outputs whatever $\mathcal{A}$ outputs.

Analysis. If $b = 0$, the challenger's value is $y^\star = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_0)))$, matching **Game** $\mathbf{1_0}$. If $b = 1$, then $y^\star = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x_1)))$, matching **Game** $\mathbf{1_1}$. Hence,

$$\big|\Pr[\textbf{Game } \mathbf{1_0} = 1] - \Pr[\textbf{Game } \mathbf{1_1} = 1]\big| \le \mathsf{Adv}^{\mathsf{PRF\text{-}Psd}}_{\mathcal{A}'}(\lambda).$$

Concluding the proof. We therefore have

$$\begin{aligned}\big|\Pr[\textbf{Game } \mathbf{0_0} = 1] - \Pr[\textbf{Game } \mathbf{0_1} = 1]\big| \le{}& \big|\Pr[\textbf{Game } \mathbf{0_0} = 1] - \Pr[\textbf{Game } \mathbf{1_0} = 1]\big| \\ &+ \big|\Pr[\textbf{Game } \mathbf{0_1} = 1] - \Pr[\textbf{Game } \mathbf{1_1} = 1]\big| \\ &+ \big|\Pr[\textbf{Game } \mathbf{1_0} = 1] - \Pr[\textbf{Game } \mathbf{1_1} = 1]\big|.\end{aligned}$$

The first two terms are negligible by the simulatability of SoK, and the last term is bounded by the PRF pseudorandomness advantage. Hence,

$$\big|\Pr[\textbf{Game } \mathbf{0_0} = 1] - \Pr[\textbf{Game } \mathbf{0_1} = 1]\big| \le \mathsf{Adv}^{\mathsf{PRF\text{-}Psd}}_{\mathcal{A}'}(\lambda) + \mathsf{negl}(\lambda),$$

which proves that the scheme satisfies linkable anonymity. □

Lemma 3. *If event P occurs, then $((\tau, f_\tau(x_i)), \cdot) \notin \mathcal{L}'_{\mathsf{PRF}}$ for all $i \in \{0, 1\}$.*

Proof. Recall that

$$\begin{aligned}D_{\tau,0} &= \{x \in \mathcal{A} \mid \exists\, \Sigma,\ (x, \cdot, \cdot, \tau, \cdot) \in \mathcal{L}_{\mathsf{Sign}} \wedge \tau \in L_x\},\\ D_{\tau,1} &= \{x \in \mathcal{A} \mid (x, \cdot, \cdot) \in \mathcal{L}_{\mathsf{Corrupt}} \wedge \tau \in L_x\}.\end{aligned}$$

The reduction $\mathcal{A}'$ queries $(\tau, f_\tau(x))$ to the PRF challenger only when x has appeared in O_{Corrupt} or in an O_{Sign} query. Thus, for such x, we have $x \in D_{\tau,0} \cup D_{\tau,1}$. Under event P, $f_\tau(x) \neq f_\tau(x_i)$ for all i, so $(\tau, f_\tau(x_i))$ is never queried to the PRF challenger. Hence $((\tau, f_\tau(x_i)), \cdot) \notin \mathcal{L}'_{\mathsf{PRF}}$ for all $i \in \{0, 1\}$, as claimed. □

5 An Instantiation of CLABS from Lattices

5.1 High-Level Overview

We now instantiate our generic CLABS framework (Section 3) under standard lattice assumptions in the QROM. Our construction combines three established ingredients: the pseudorandom function in [19] (mentioned in Section A.3), the general-lattice JRS signature [26], and a zero-knowledge layer derived from the abstract protocol of Yang et al. [47]. This combination yields the first CLABS realization secure against quantum adversaries, supporting arbitrary Boolean circuits as signing policies.

Core ideas. The authority evaluates the PRF to derive deterministic pseudorandom tokens $\mathbf{y}_{x,\tau} = \mathsf{PRF}_K((\tau, f_\tau(x)))$ and certifies their correctness through a short JRS signature on $(x, L_x, \{\mathbf{y}_{x,\tau}\}_{\tau \in L_x})$. During signing, the user produces a signature of knowledge (SoK) attesting that: (i) it possesses a valid JRS certificate on its registered attribute; (ii) its attribute satisfies the policy circuit $C(x) = 1$; and (iii) the published tag $\mathbf{t}$ equals $\mathbf{y}_{x,\tau}$ when $\tau \in L_x$ and $\mathbf{0}$ otherwise. All these relations are merged into one lattice-based zero-knowledge relation à la Yang et al.

Parameter choices. Let λ be the security parameter. We set $n = \tilde{O}(\lambda)$, $q = \mathrm{poly}(\lambda)$, $m = O(n \log q)$, and let $p < q$ be the LWR modulus. The JRS signature uses tag dimension $d = \Theta(\log q)$, and the number of linkable contexts per user is fixed to $k = \mathrm{poly}(\lambda)$. These settings ensure hardness of SIS and $\mathsf{LWE}/\mathsf{LWR}$ at level $2^{\tilde{\Omega}(\lambda)}$.

5.2 The Zero-Knowledge Layer

We next show how the full CLABS signing relation—policy satisfaction, certificate validity, and tag consistency—can be jointly proven using a lattice-based zero-knowledge argument. Our objective is to represent all these verification conditions within a single algebraic framework that allows efficient proofs over structured linear and multiplicative relations.

Background: Yang et al.'s General Relation. We build upon the framework of Yang et al. [47], who proposed a generic lattice-based relation of the form

$$\mathbf{A}\mathbf{x} = \mathbf{y} \pmod q, \qquad \forall (h,i,j) \in \mathcal{M} : \ \mathbf{x}[h] = \mathbf{x}[i] \cdot \mathbf{x}[j],$$

where $\mathbf{A} \in \mathbb{Z}_q^{r \times n}$, $\mathbf{y} \in \mathbb{Z}_q^r$, and $\mathcal{M}$ is a collection of coordinate triples specifying componentwise multiplicative constraints. They designed a public-coin Σ-protocol for proving knowledge of such a witness, which is complete, special-sound, and *computationally zero-knowledge* under standard lattice assumptions: special soundness follows from the hardness of the SIS problem, while zero-knowledge is ensured by the LWE assumption. A non-interactive version is obtained by applying the Fiat–Shamir transform (in the ROM) or the Unruh transform (in the QROM).

Reduction to Yang et al.'s Relation. The CLABS signing relation naturally involves both linear and multiplicative constraints: (i) the correctness of the NAND-based policy circuit, (ii) the validity of the JRS certificate, and (iii) the consistency of the context-dependent link tag. To enable a unified proof, we compile all these constraints into the *abstract lattice relation* of Yang et al. [47], where each condition is expressed either as a linear equation in the witness or as a multiplicative equality of the form $\mathbf{x}[h] = \mathbf{x}[i]\mathbf{x}[j]$ in $\mathbb{Z}_q$.

In this encoding, every linear or arithmetic constraint in CLABS is represented as a row of a public matrix $\mathbf{A}_{\mathsf{clabs}}$ with corresponding right-hand side

$\mathbf{y}_{\mathsf{clabs}}$, while each product (e.g., NAND multiplication, bit-squareness, tag–selector interaction) is represented as an index triple in $\mathcal{M}_{\mathsf{clabs}}$. For example, each NAND gate contributes a linear constraint $z+t_g-\mathbf{x}_{\mathrm{one}}=0$ and a product triple (t_g,a,b); the JRS verification relation $\mathbf{A}\mathbf{v}_1+\mathbf{w}-\mathbf{B}\mathbf{v}_2=\mathbf{u}+\mathbf{D}\mathbf{m}$ is linear, while its tag equation $\mathbf{w}=\boldsymbol{\vartheta}\odot\mathbf{G}_{\mathrm{blk}}\mathbf{v}_2$ is captured through d multiplicative triples. Similarly, tag selection $\mathbf{t}[j]=\sum_i s_i\tilde{\mathbf{y}}_{x,\tau_i}[j]$ is expanded into linear rows together with auxiliary product constraints $u_{i,j}=s_i\tilde{\mathbf{y}}_{x,\tau_i}[j]$.

Formal Relation Definition. We denote by

$$R_{\mathsf{clabs}}=\Big\{\ ((\mathbf{A}_{\mathsf{clabs}},\mathbf{y}_{\mathsf{clabs}},\mathcal{M}_{\mathsf{clabs}}),\mathbf{x}_{\mathsf{clabs}})\ \Big|\ \mathbf{A}_{\mathsf{clabs}}\mathbf{x}_{\mathsf{clabs}}=\mathbf{y}_{\mathsf{clabs}}\pmod q,\\ \forall(h,i,j)\in\mathcal{M}_{\mathsf{clabs}}:\ \mathbf{x}_{\mathsf{clabs}}[h]=\mathbf{x}_{\mathsf{clabs}}[i]\cdot\mathbf{x}_{\mathsf{clabs}}[j]\Big\}.$$

A witness $\mathbf{x}_{\mathsf{clabs}}$ satisfies R_{clabs} if and only if it encodes (i) a certified user attribute and valid JRS signature, (ii) a satisfying policy assignment $C(x)=1$, and (iii) a correctly formed tag consistent with the linking context. Thus, $(\mathbf{A}_{\mathsf{clabs}},\mathbf{y}_{\mathsf{clabs}},\mathcal{M}_{\mathsf{clabs}})$ serves as the public instance of the unified lattice relation, and R_{clabs} defines the exact set of valid CLABS signing witnesses.

A proof of knowledge for this unified instance can then be generated using the lattice-based Σ-protocol of Yang et al. [47], and subsequently converted into a non-interactive argument in the ROM [5] or QROM [9,13] via the Fiat–Shamir [20] or Unruh transform [44], respectively. Further details appear in Appendix B.

5.3 QROM-Secure Signature of Knowledge via Unruh's Transform

Given the public instance $(\mathbf{A}_{\mathsf{clabs}},\mathbf{y}_{\mathsf{clabs}},\mathcal{M}_{\mathsf{clabs}})$ defining the relation R_{clabs}, our goal is to construct a post-quantum secure SoK proving that the signer knows a valid witness $\mathbf{x}_{\mathsf{clabs}}$ such that $((\mathbf{A}_{\mathsf{clabs}},\mathbf{y}_{\mathsf{clabs}},\mathcal{M}_{\mathsf{clabs}}),\mathbf{x}_{\mathsf{clabs}})\in R_{\mathsf{clabs}}$, and that this knowledge is bound to a message M.

Setup. Let H_1,H_2 be independent quantum-accessible random oracles. Given a message M and a valid witness $\mathbf{x}_{\mathsf{clabs}}$, the signer and verifier proceed as follows.

- **Signer:** Execute the first step of the underlying Σ-protocol to obtain a commitment com. Query $H_1(\mathsf{com},M)$ to derive a multi-challenge vector $\mathbf{ch}=(ch_1,\ldots,ch_\kappa)$. (Including M in $H_1(\mathsf{com},M)$ ensures *message binding*: each signature is uniquely tied to the message and cannot be reused for a different one.) For each challenge ch_i, compute the corresponding response rsp_i and digest $h_i=H_2(\mathsf{rsp}_i)$. Output the signature of knowledge
$$\pi=(\mathsf{com},\{(ch_i,h_i,\mathsf{rsp}_i)\}_{i=1}^{\kappa}).$$
- **Verifier:** On input (M,π), recompute $\mathbf{ch}=H_1(\mathsf{com},M)$, check $h_i=H_2(\mathsf{rsp}_i)$ for all i, and verify that each tuple $(\mathsf{com},ch_i,\mathsf{rsp}_i)$ satisfies the Yang-verifier equations for the public instance $(\mathbf{A}_{\mathsf{clabs}},\mathbf{y}_{\mathsf{clabs}},\mathcal{M}_{\mathsf{clabs}})$.

Security properties. Under the SIS and LWE assumptions in the QROM, the SoK satisfies:

Correctness. If the signer holds a valid witness $\mathbf{x}_{\mathsf{clabs}}$ such that

$$((\mathbf{A}_{\mathsf{clabs}}, \mathbf{y}_{\mathsf{clabs}}, \mathcal{M}_{\mathsf{clabs}}), \mathbf{x}_{\mathsf{clabs}}) \in R_{\mathsf{clabs}},$$

then the verifier accepts $\pi \leftarrow \mathsf{SoK.Sign}(pp, (\mathbf{A}_{\mathsf{clabs}}, \mathbf{y}_{\mathsf{clabs}}, \mathcal{M}_{\mathsf{clabs}}, M), \mathbf{x}_{\mathsf{clabs}})$ with overwhelming probability.

Simulatability. There exists a quantum-polynomial-time simulator that produces signatures indistinguishable from real ones without knowing any witness, by adaptively programming H_1, H_2. This property follows from the computational zero-knowledge of Yang *et al.*'s protocol under the LWE assumption, ensuring privacy of the signer's certified attribute and secret key.

Extractability. There exists a quantum-polynomial-time extractor that, given oracle access to any adversary producing valid SoK signatures with non-negligible probability, extracts a corresponding witness $\mathbf{x}_{\mathsf{clabs}}$. This extraction guarantee is inherited from the special-structure properties of the underlying Σ-protocol under the SIS assumption.

Consequently, the SoK instantiated via Unruh's transform constitutes a post-quantum secure signature of knowledge in the QROM, supporting all verification constraints embedded in R_{clabs}. It satisfies correctness, simulatability, and extractability under standard lattice assumptions and forms the basis of our lattice-based CLABS construction.

5.4 Description of Our CLABS Scheme

We now describe the concrete lattice-based instantiation of our CLABS construction. It integrates the lattice-based PRF from Section A.3, the general-lattice JRS signature [26], and the Unruh-transformed SoK described in Section 5.3. All algorithms operate over $\mathbb{Z}_q$ for a modulus $q = \mathrm{poly}(\lambda)$.

Setup.

1. Choose parameters (n, m), moduli (p, q), and Gaussian bound B. Fix a hash function $\mathsf{H}_{\mathrm{mat}} \colon \{0,1\}^* \to \mathbb{Z}_q^{m \times n}$. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ and define the lattice-based PRF
$$\mathsf{PRF}_K(v) = \left\lfloor \mathsf{H}_{\mathrm{mat}}(v)\mathbf{s} \right\rceil_p \in \mathbb{Z}_p^m.$$
2. Run the JRS key-generation algorithm $(\mathsf{vk}_A, \mathsf{sk}_A) \leftarrow \mathsf{JRS.KeyGen}(1^\lambda)$, obtaining
$$\mathsf{vk}_A = (\mathbf{A}, \mathbf{B}, \mathbf{D}, \mathbf{G}), \quad \mathsf{sk}_A = (\mathbf{R}_1, \mathbf{R}_2),$$
where $\mathbf{A}, \mathbf{B}, \mathbf{D}, \mathbf{G} \in \mathbb{Z}_q^{n \times m}$ are public random matrices and $\mathbf{R}_1, \mathbf{R}_2 \in \mathbb{Z}_q^{m \times m}$ are short trapdoor matrices used for opening commitments in the JRS signature.
3. Sample a public matrix for the zero-knowledge layer, $\mathbf{A}_{\mathsf{SoK}} \leftarrow \mathbb{Z}_q^{r \times n'}$.

4. Publish
$$\mathsf{pp} = (\lambda, q, p, B, n, m, k, \mathsf{vk}_A, \mathsf{H}_{\mathrm{mat}}, \mathbf{A}_{\mathsf{SoK}}, \mathcal{F}),$$
and keep $\mathsf{msk} = (K, \mathsf{sk}_A)$ secret.

Key generation. For a user with attribute x and authorized tag list $L_x = \{\tau_1, \ldots, \tau_k\}$:

1. For each $\tau_i \in L_x$, compute $\mathbf{v}_{x,\tau_i} = (\tau_i, f_{\tau_i}(x))$ and
$$\mathbf{y}_{x,\tau_i} = \mathsf{PRF}_K(\mathbf{v}_{x,\tau_i}) = \left\lfloor \mathsf{H}_{\mathrm{mat}}(\mathbf{v}_{x,\tau_i})\mathbf{s} \right\rceil_p \in \mathbb{Z}_p^m.$$
Each $\mathbf{y}_{x,\tau_i}$ acts as a deterministic tag commitment.
2. Form the message $m_x = (x, \{(\tau_i, \mathbf{y}_{x,\tau_i})\}_{i=1}^k)$ and sign it with the authority's key:
$$\sigma_A \leftarrow \mathsf{JRS.Sign}(\mathsf{sk}_A, m_x).$$
In the JRS scheme, this signature is a tuple
$$\sigma_A = (\mathbf{u}, \mathbf{v}_1, \mathbf{v}_2, \mathbf{w}),$$
where
$$\begin{aligned} \mathbf{u} &= \mathbf{A}\mathbf{v}_1 + \mathbf{w} - \mathbf{B}\mathbf{v}_2 - \mathbf{D}\mathbf{m}_x \pmod q, \\ \mathbf{w} &= \mathbf{G}\mathbf{v}_2 + \boldsymbol{\vartheta} \pmod q, \end{aligned}$$
for short vectors $\mathbf{v}_1, \mathbf{v}_2, \boldsymbol{\vartheta}$ with $\|\mathbf{v}_i\|, \|\boldsymbol{\vartheta}\| \le B$. Verification of σ_A checks these linear equations modulo q.
3. Output the user's secret key:
$$sk_{x,L_x} = (x, L_x, \{\mathbf{y}_{x,\tau_i}\}_{i=1}^k, \sigma_A).$$

Signing. Given sk_{x,L_x}, a message M, a circuit C, and a context τ:

1. Compute the public tag
$$\mathbf{t} = \begin{cases} \mathbf{y}_{x,\tau}, & \text{if } \tau \in L_x, \\ \mathbf{0}^m, & \text{otherwise.} \end{cases}$$
2. Construct the lattice relation R_{clabs} as defined in Section 5.2, which enforces the following public conditions:
   - correctness of the JRS signature σ_A;
   - satisfaction of $C(x) = 1$;
   - correct computation of $\mathbf{t}$ from the PRF outputs.
3. Let $\mathsf{SoK.Sign}$ and $\mathsf{SoK.Verify}$ denote the signing and verification algorithms of the Unruh-transformed SoK (Section 5.3). Run the signer to obtain
$$\pi \leftarrow \mathsf{SoK.Sign}\big(pp_{\mathsf{SoK}}, (R_{\mathsf{clabs}}, M), \mathbf{x}_{\mathsf{clabs}}\big),$$
where the witness $\mathbf{x}_{\mathsf{clabs}}$ includes $(x, L_x, \mathbf{s}, \mathbf{v}_1, \mathbf{v}_2, \boldsymbol{\vartheta})$.
4. Output the signature
$$\Sigma = (M, C, \tau, \mathbf{t}, \pi).$$

Verification. On input (M, C, Σ, τ), parse $\Sigma = (M, C, \tau, \mathbf{t}, \pi)$ and accept iff

$$\mathsf{SoK.Verify}(pp_{\mathsf{SoK}}, (R_{\mathsf{clabs}}, M), \pi) = 1.$$

Verification thus checks the validity of the Unruh-based SoK, which implicitly re-verifies the JRS certificate and policy satisfaction.

Linking. Given $(\Sigma_0, \Sigma_1, \tau)$, output

$$\mathsf{Link}(\mathsf{pp}, \Sigma_0, \Sigma_1, \tau) = \begin{cases} 1, & \text{if } \mathbf{t}_0 = \mathbf{t}_1 \neq \mathbf{0}^m, \\ 0, & \text{otherwise.} \end{cases}$$

Two signatures are linkable if they share the same nonzero tag vector derived from the PRF output $\mathsf{PRF}_K((\tau, f_\tau(x)))$.

5.5 Analyses

Signature size. Let us evaluate the signature size based on the security parameter λ, the number of linkable contexts k, the tag length m, and the number N of NAND gates in the policy circuit C. A signature is $\Sigma = (M, C, \tau, \mathbf{t}, \pi)$, where $\mathbf{t} \in \mathbb{Z}_p^m$ and π is the Unruh-transformed zero-knowledge component.

The ZK protocol of [47] has inverse-polynomial knowledge error and must be repeated $\rho = O(\lambda / \log \lambda)$ times to obtain negligible error. Each base proof attests to $O(N + k\,m + \lambda)$ linear and multiplicative constraints, so a single instance has size $\tilde{O}(N + k\,m + \lambda)$. After ρ repetitions, the overall proof size becomes

$$|\pi| = \tilde{O}\big(\lambda(N + k\,m + \lambda)\big).$$

Including the tag $\mathbf{t}$ contributes only $O(m)$ additional elements, which is asymptotically dominated. Therefore, the total signature size is

$$|\Sigma| = O(m) + \tilde{O}\big(\lambda(N + k\,m + \lambda)\big) = \tilde{O}\big(\lambda(N + k\,m + \lambda)\big).$$

Security properties. Our lattice-based CLABS inherits the security properties of the employed cryptographic building blocks, i.e., the JRS signature [26], the LWR-based PRF [19] and the Unruh-transformed SoK from Section 5.3, inspired by Yang et al.'s zero-knowledge protocol [47].

The auxiliary algorithms $\mathsf{SimSetup}$ and $\mathsf{Extract}$, which are required for security analyses, work as follows: (i) $\mathsf{SimSetup}(1^\lambda)$ generates simulated parameters $(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr})$ with preprogrammed quantum oracles (H_1, H_2), allowing simulated proofs indistinguishable from real ones; and (ii) $\mathsf{Extract}(\mathsf{pp}_{\mathsf{SoK}}, \mathsf{tr}, \cdot)$ rewinds a successful adversary to recover a valid witness $\mathbf{x}'_{\mathsf{clabs}}$ satisfying $\mathbf{A}_{\mathsf{clabs}} \cdot \mathbf{x}'_{\mathsf{clabs}} = \mathbf{y}_{\mathsf{clabs}} \pmod q$.

In summary, under the standard SIS and LWE/LWR assumptions, our construction achieves in the QROM all six desired security properties of Section 4:

– **Correctness:** Guaranteed by the completeness of the JRS signature and of the Unruh-transformed SoK, together with deterministic PRF evaluation. The LWR rounding ensures that $\Pr[\mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, X) = \mathbf{0}^m]$ is negligible, yielding consistent verification.
– **Setup indistinguishability:** The simulated setup produces parameters computationally indistinguishable from the real ones under the LWE assumption, even for quantum adversaries.
– **Linkability:** Deterministic and collision-resistant PRF outputs ensure that two signatures can be linked iff they originate from the same certified attribute (and context). Together with SoK extractability, this guarantees consistent linking.
– **Unforgeability:** Every valid CLABS signature includes a genuine JRS certificate σ_A and a valid SoK component π. By SoK extractability and the SIS-based unforgeability of JRS, any successful forgery would yield a new valid JRS signature, contradicting the SIS assumption.
– **Unlinkable anonymity:** When $\tau \notin L_x$, the tag $\mathbf{t} = \mathbf{0}^m$ and the simulated proof (via $\mathsf{SimSetup}$) make real and simulated signatures indistinguishable under the LWE-based zero-knowledge property.
– **Linkable anonymity:** The tag $\mathbf{t} = \mathsf{PRF.Eval}(\mathsf{pp}_{\mathsf{PRF}}, K, (\tau, f_\tau(x)))$, computed when $\tau \in L_x$, is pseudorandom under LWR, while SoK simulatability hides the witness. Thus, signatures of different users remain indistinguishable except when their tags coincide, as required by Definition 6.

References

1. Joël Alwen, Stephan Krenn, Krzysztof Pietrzak, and Daniel Wichs. Learning with rounding, revisited. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013*, volume 8042 of *LNCS*, pages 57–74. Springer, 2013.
2. Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, April 2012.
3. Rachid El Bansarkhani and Ali El Kaafarani. Post-quantum attribute-based signatures from lattice assumptions. IACR Cryptol. ePrint Arch. 2016/823, 2016.
4. Mihir Bellare and Georg Fuchsbauer. Policy-based signatures. In *Public-Key Cryptography - PKC 2014*, volume 8383 of *LNCS*, pages 520–537. Springer, 2014.
5. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
6. Osman Biçer and Alptekin Küpçü. Versatile ABS: usage limited, revocable, threshold traceable, authority hiding, decentralized attribute based signatures. IACR Cryptol. ePrint Arch. 2019/203, 2019.
7. Andrej Bogdanov, Siyao Guo, Daniel Masny, Silas Richelson, and Alon Rosen. On the hardness of learning with rounding over small modulus. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part I*, volume 9562 of *LNCS*, pages 209–224. Springer, Berlin, Heidelberg, January 2016.

8. Alexandra Boldyreva and Daniele Micciancio, editors. *CRYPTO 2019, Part I*, volume 11692 of *LNCS*. Springer, Cham, August 2019.
9. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.
10. Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 575–584. ACM Press, June 2013.
11. Shantian Cheng, Khoa Nguyen, and Huaxiong Wang. Policy-based signature scheme from lattices. *Des. Codes Cryptogr.*, 81(1):43–74, 2016.
12. Pratish Datta, Tatsuaki Okamoto, and Katsuyuki Takashima. Efficient attribute-based signatures for unbounded arithmetic branching programs. In Dongdai Lin and Kazue Sako, editors, *PKC 2019*, volume 11442 of *LNCS*, pages 127–158. Springer, 2019.
13. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383. Springer, Cham, August 2019.
14. Constantin Catalin Dragan, Daniel Gardham, and Mark Manulis. Hierarchical attribute-based signatures. In Jan Camenisch and Panos Papadimitratos, editors, *CANS 18*, volume 11124 of *LNCS*, pages 213–234. Springer, Cham, September / October 2018.
15. Ali El Kaafarani, Liqun Chen, Essam Ghadafi, and James H. Davenport. Attribute-based signatures with user-controlled linkability. In Dimitris Gritzalis, Aggelos Kiayias, and Ioannis G. Askoxylakis, editors, *CANS 14*, volume 8813 of *LNCS*, pages 256–269. Springer, Cham, October 2014.
16. Ali El Kaafarani, Essam Ghadafi, and Dalia Khader. Decentralized traceable attribute-based signatures. In Josh Benaloh, editor, *CT-RSA 2014*, volume 8366 of *LNCS*, pages 327–348. Springer, Cham, February 2014.
17. Ali El Kaafarani and Shuichi Katsumata. Attribute-based signatures for unbounded circuits in the ROM and efficient instantiations from lattices. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018*, volume 10770 of *LNCS*, pages 89–119. Springer, 2018.
18. Alex Escala, Javier Herranz, and Paz Morillo. Revocable attribute-based signatures with adaptive security in the standard model. In Abderrahmane Nitaj and David Pointcheval, editors, *AFRICACRYPT 11*, volume 6737 of *LNCS*, pages 224–241. Springer, Berlin, Heidelberg, July 2011.
19. Hanwen Feng, Jianwei Liu, Qianhong Wu, and Ya-Nan Li. Traceable ring signatures with post-quantum security. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 442–468. Springer, Cham, February 2020.
20. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
21. Marc Fischlin and Cristina Onete. Relaxed security notions for signatures of knowledge. In Javier Lopez and Gene Tsudik, editors, *ACNS 2011*, volume 6715 of *LNCS*, pages 309–326. Springer, Berlin, Heidelberg, June 2011.
22. Daniel Gardham and Mark Manulis. Revocable hierarchical attribute-based signatures from lattices. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 459–479. Springer, 2022.

23. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
24. Jens Groth and Mary Maller. Snarky signatures: Minimal signatures of knowledge from simulation-extractable SNARKs. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 581–612. Springer, Cham, August 2017.
25. Javier Herranz. Attribute-based signatures from RSA. *Theor. Comput. Sci.*, 527:73–82, 2014.
26. Corentin Jeudy, Adeline Roux-Langlois, and Olivier Sanders. Lattice signature with efficient protocols, application to anonymous credentials. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 351–383. Springer, Cham, August 2023.
27. Jin Li, Man Ho Au, Willy Susilo, Dongqing Xie, and Kui Ren. Attribute-based signature and its applications. In Dengguo Feng, David A. Basin, and Peng Liu, editors, *ASIACCS 10*, pages 60–69. ACM Press, April 2010.
28. Yanling Lian, Li Xu, and Xinyi Huang. Attribute-based signatures with efficient revocation. In *INCoS 2013*, pages 573–577. IEEE, 2013.
29. Benoît Libert, Khoa Nguyen, Alain Passelègue, and Radu Titiu. Simulation-sound arguments for LWE and applications to KDM-CCA2 security. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 128–158. Springer, Cham, December 2020.
30. San Ling, Khoa Nguyen, Duong Hieu Phan, Khai Hanh Tang, Huaxiong Wang, and Yanhong Xu. Fully dynamic attribute-based signatures for circuits from codes. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part I*, volume 14601 of *LNCS*, pages 37–73. Springer, Cham, April 2024.
31. Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. Linkable spontaneous anonymous group signature for ad hoc groups. In *ISP 2004*, volume 3108 of *LNCS*, pages 325–335. Springer, 2004.
32. Hemanta K. Maji, Manoj Prabhakaran, and Mike Rosulek. Attribute-based signatures. In Aggelos Kiayias, editor, *CT-RSA 2011*, volume 6558 of *LNCS*, pages 376–392. Springer, Berlin, Heidelberg, February 2011.
33. Khoa Nguyen, Fuchun Guo, Willy Susilo, and Guomin Yang. Multimodal private signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 792–822. Springer, Cham, August 2022.
34. Tatsuaki Okamoto and Katsuyuki Takashima. Efficient attribute-based signatures for non-monotone predicates in the standard model. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 35–52. Springer, Berlin, Heidelberg, March 2011.
35. Tatsuaki Okamoto and Katsuyuki Takashima. Decentralized attribute-based signatures. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 125–142. Springer, 2013.
36. Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Boldyreva and Micciancio [8], pages 89–114.
37. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.
38. Yusuke Sakai, Nuttapong Attrapadung, and Goichiro Hanaoka. Attribute-based signatures for circuits from bilinear map. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016*, volume 9614 of *LNCS*, pages 283–300. Springer, 2016.

39. Yusuke Sakai, Shuichi Katsumata, Nuttapong Attrapadung, and Goichiro Hanaoka. Attribute-based signatures for unbounded languages from standard assumptions. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 493–522. Springer, Cham, December 2018.
40. Qianqian Su, Rui Zhang, Rui Xue, and Pengchao Li. Revocable attribute-based signature for blockchain-based healthcare system. *IEEE Access*, 8:127884–127896, 2020.
41. Nam Tran, Khoa Nguyen, Dongxi Liu, Josef Pieprzyk, and Willy Susilo. Improved multimodal private signatures from lattices. In Tianqing Zhu and Yannan Li, editors, *ACISP 24, Part II*, volume 14896 of *LNCS*, pages 3–23. Springer, Singapore, July 2024.
42. Nam Tran, Khoa Nguyen, Dongxi Liu, Josef Pieprzyk, and Willy Susilo. Lattice-based group signatures in the standard model, revisited. In *ASIACRYPT 2025*, volume 16248 of *LNCS*, pages 70–102. Springer, 2025.
43. Rotem Tsabary. An equivalence between attribute-based signatures and homomorphic signatures, and new constructions for both. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part II*, volume 10678 of *LNCS*, pages 489–518. Springer, Cham, November 2017.
44. Dominique Unruh. Non-interactive zero-knowledge proofs in the quantum random oracle model. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 755–784. Springer, Berlin, Heidelberg, April 2015.
45. Miguel Urquidi, Dalia Khader, Jean Lancrenon, and Liqun Chen. Attribute-based signatures with controllable linkability. In Moti Yung, Jian-biao Zhang, and Zhen Yang, editors, *INTRUST 2015*, volume 9565 of *LNCS*, pages 114–129. Springer, 2015.
46. Brent Waters. A new approach for non-interactive zero-knowledge from learning with errors. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *56th ACM STOC*, pages 399–410. ACM Press, June 2024.
47. Rupeng Yang, Man Ho Au, Zhenfei Zhang, Qiuliang Xu, Zuoxia Yu, and William Whyte. Efficient lattice-based zero-knowledge arguments with standard soundness: Construction and applications. In Boldyreva and Micciancio [8], pages 147–175.
48. Tsz Hon Yuen, Joseph K. Liu, Xinyi Huang, Man Ho Au, Willy Susilo, and Jianying Zhou. Forward secure attribute-based signatures. In Tat Wing Chim and Tsz Hon Yuen, editors, *ICICS 2012*, volume 7618 of *LNCS*, pages 167–177. Springer, 2012.

A Cryptographic Building Blocks

In this section, we recall the syntax and security properties of three fundamental primitives used in our CLABS construction: signature schemes, signatures of knowledge, and pseudorandom functions. We then summarize the lattice-based hardness assumptions on which our constructions rely.

A.1 Signature Scheme

A digital signature scheme allows a user to authenticate messages using a secret key, such that any verifier can publicly confirm the message's authenticity using the corresponding public key.

Definition 7 (Signature Scheme). *Let $\ell(\lambda)$ be a polynomially bounded function of the security parameter λ. A signature scheme is a tuple of PPT algorithms*

$$\mathsf{Sig} = (\mathsf{Sig.Setup}, \mathsf{Sig.KeyGen}, \mathsf{Sig.Sign}, \mathsf{Sig.Verify}),$$

defined as follows:

$\mathsf{Sig.Setup}(1^\lambda) \to \mathsf{pp}$**:** *On input the security parameter λ, outputs public parameters* pp.

$\mathsf{Sig.KeyGen}(\mathsf{pp}) \to (\mathsf{pk}, \mathsf{sk})$**:** *On input* pp*, outputs a public/secret key pair* $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{Sig.Sign}(\mathsf{sk}, M) \to \Sigma$**:** *On input a secret key* sk *and a message $M \in \{0,1\}^{\ell(\lambda)}$, outputs a signature Σ.*

$\mathsf{Sig.Verify}(\mathsf{pk}, M, \Sigma) \to \{0,1\}$**:** *On input a public key* pk*, a message M, and a signature Σ, outputs* 1 *if Σ is valid and* 0 *otherwise.*

Correctness. A signature scheme is *correct* if any honestly generated key pair produces verifiable signatures.

Definition 8 (Correctness). *For all $\lambda \in \mathbb{N}$ and $M \in \{0,1\}^{\ell(\lambda)}$, let* $\mathsf{pp} \leftarrow \mathsf{Sig.Setup}(1^\lambda)$ *and* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Sig.KeyGen}(\mathsf{pp})$*. Then correctness requires*

$$\mathsf{Sig.Verify}(\mathsf{pk}, M, \mathsf{Sig.Sign}(\mathsf{sk}, M)) = 1.$$

Unforgeability. No efficient adversary should be able to forge a valid signature on a new message that was not signed before.

Definition 9 (Unforgeability). *A signature scheme is* existentially unforgeable under adaptive chosen-message attacks *(EUF-CMA) if for every PPT adversary $\mathcal{A}$,*

$$\mathbf{Adv}_{\mathcal{A}}^{\mathsf{Sig\text{-}UF}}(\lambda) = \Pr\left[\mathbf{Exp}_{\mathcal{A}}^{\mathsf{Sig\text{-}UF}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{Sig\text{-}UF}}(\lambda)$ is described below.

$\mathsf{pp} \leftarrow \mathsf{Sig.Setup}(1^\lambda)$.
$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Sig.KeyGen}(\mathsf{pp})$.
$(M, \Sigma) \leftarrow \mathcal{A}^{\mathsf{Sign}(\mathsf{sk}, \cdot)}(\mathsf{pk})$.
if $(M, \Sigma) \notin Q \ \wedge \ \mathsf{Sig.Verify}(\mathsf{pk}, M, \Sigma) = 1$ **then**
  return 1.
return 0.

Fig. 5: Experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{Sig-UF}}(\lambda)$. In the experiment, Q denotes the list of (M, Σ) s.t M has been queried to $\mathsf{Sign}(\mathsf{sk}, \cdot)$ and Σ is the signature returned by the query..

A.2 Signature of Knowledge

A *signature of knowledge* (SoK) enables a signer to prove possession of a valid witness for an NP-relation R, while keeping the witness hidden. Any verifier can check the signature using only the public instance. Our definition follows the formalizations in [21,24].

Definition 10 (Signature of Knowledge). *Let $R \subseteq \{0,1\}^{N(\lambda)} \times \{0,1\}^{N(\lambda)}$ be a polynomial-time relation, and let $\ell(\lambda)$ be polynomial in λ. A signature of knowledge scheme consists of the PPT algorithms*

$$\mathsf{SoK} = (\mathsf{SoK.Setup}, \mathsf{SoK.Sign}, \mathsf{SoK.Verify}, \mathsf{SoK.SimSetup}, \mathsf{SoK.SimSign}, \mathsf{SoK.Extract}),$$

defined as follows:

$\mathsf{SoK.Setup}(1^\lambda, R) \to \mathsf{pp}$**:** *Outputs public parameters.*
$\mathsf{SoK.Sign}(\mathsf{pp}, R, M, x, w) \to \sigma$**:** *Given* pp*, R, message M, instance x, and witness w with $(x, w) \in R$, outputs a signature σ.*
$\mathsf{SoK.Verify}(\mathsf{pp}, R, M, x, \sigma) \to \{0,1\}$**:** *Verifies that σ is valid for (M, x).*
$\mathsf{SoK.SimSetup}(1^\lambda, R) \to (\mathsf{pp}, \mathsf{tr})$**:** *Outputs simulated parameters and trapdoor* tr*.*
$\mathsf{SoK.SimSign}(\mathsf{pp}, R, \mathsf{tr}, M, x) \to \sigma$**:** *Generates a simulated signature without knowing w.*
$\mathsf{SoK.Extract}(\mathsf{pp}, R, \mathsf{tr}, M, x, \sigma) \to w$**:** *Extracts a witness from a valid signature using the trapdoor.*

A secure signature of knowledge must satisfy four properties: correctness, extractability, simulatability, and parameter indistinguishability.

Correctness. A signature scheme is *correct* if any vaild instance-message pair and message gives a valid signature.

Definition 11 (Correctness). *For every $(x, w) \in R$, message $M \in \{0,1\}^{\ell(\lambda)}$, and parameters $\mathsf{pp} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)$, if $\sigma \leftarrow \mathsf{SoK.Sign}(\mathsf{pp}, R, M, x, w)$, then $\mathsf{SoK.Verify}(\mathsf{pp}, R, M, x, \sigma) = 1$.*

Extractability. Intuitively, if an adversary produces a new valid signature that was not obtained via the simulator, then an extractor using the trapdoor must be able to recover a valid witness.

Definition 12 (Extractability). *A signature of knowledge provides* extractability *if for every PPT adversary $\mathcal{A}$,*

$$\mathbf{Adv}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathbf{Exp}^{\mathsf{SoK\text{-}Ext}}_{\mathcal{A}}$ is defined below.

$(\mathsf{pp}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R)$.

$(x, M, \sigma) \leftarrow \mathcal{A}^{\mathsf{Sim}(\mathsf{pp},\mathsf{tr},\cdot,\cdot,\cdot)}(\mathsf{pp})$.

$w \leftarrow \mathsf{SoK.Extract}(\mathsf{pp}, R, \mathsf{tr}, x, M, \Sigma)$.

if $(x, M, \sigma) \notin Q \ \wedge \ \mathsf{Sig.Verify}(\mathsf{pk}, R, x, M, \sigma) = 1 \ \wedge \ (x, w) \notin R$ **then**

  return 1.

return 0.

Fig. 6: Experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{SoK-Ext}}(\lambda)$. In the experiment, $\mathsf{Sim}(\mathsf{pp}, \mathsf{tr}, \cdot, \cdot, \cdot)$ is a query that: On input (x, w, M), return $\perp$ if $(x, w) \notin R$. Otherwise return $\mathsf{SoK.SimSign}(\mathsf{pp}, R, \mathsf{tr}, x, M)$. In addition, Q denotes the list of (x, M, σ) s.t $(x, M, \cdot)$ has been queried to $\mathsf{Sim}(\mathsf{pp}, R, \mathsf{tr}, \cdot, \cdot, \cdot)$ and σ is the signature returned by the query.

Simulatability. The adversary should not be able to distinguish real signatures from simulated ones.

Definition 13 (Simulatability). *For every PPT adversary $\mathcal{A}$,*

$$\Big|\Pr[\mathcal{A}^{\mathsf{P}(\mathsf{pp},\cdot,\cdot,\cdot)}(\mathsf{pp}) = 1 \mid \ \mathsf{pp} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)] - \Pr[\mathcal{A}^{\mathsf{Sim}(\mathsf{pp},\mathsf{tr},\cdot,\cdot,\cdot)}(\mathsf{pp}) = 1 \mid (\mathsf{pp}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R)]\Big| \leq \mathsf{negl}(\lambda),$$

where the oracle $\mathsf{P}(\mathsf{pp}, \cdot, \cdot, \cdot)$ outputs $\perp$ if $(x, w) \notin R$, and otherwise returns $\mathsf{SoK.Sign}(\mathsf{pp}, R, M, x, w)$. The oracle Sim behaves as in Definition 12.

Parameter Indistinguishability. Setup and simulated setups must be computationally indistinguishable.

Definition 14 (Parameter Indistinguishability). *For every PPT adversary $\mathcal{A}$ the following hold:*

$$\Big| \ \Pr[\mathcal{A}(\mathsf{pp}) = 1 \mid \mathsf{pp} \leftarrow \mathsf{SoK.Setup}(1^\lambda, R)] \ - \ \Pr[\mathcal{A}(\mathsf{pp}) = 1 \mid (\mathsf{pp}, \mathsf{tr}) \leftarrow \mathsf{SoK.SimSetup}(1^\lambda, R)] \ \Big| \leq \mathsf{negl}(\lambda).$$

A.3 Pseudorandom Function

A pseudorandom function (PRF) is a keyed deterministic function that behaves like a random function to any efficient adversary without the secret key.

Definition 15 (Pseudorandom Function). *Let $\mathcal{X}$ and $\mathcal{Y}$ be the input and output spaces. A PRF family* PRF *consisting of three algorithms defined as follows:*

$\mathsf{PRF.Setup}(1^\lambda) \to \mathsf{pp}$**:** *Outputs public parameters.*
$\mathsf{PRF.KeyGen}(\mathsf{pp}) \to K$**:** *Outputs a secret key K.*
$\mathsf{PRF.Eval}(\mathsf{pp}, K, X) \to Y$**:** *On input $X \in \mathcal{X}$, outputs $Y \in \mathcal{Y}$.*

Weak Pseudorandomness. We adopt a relaxed notion where the adversary cannot query the challenge inputs.

Definition 16 (Weak Pseudorandomness). *A PRF satisfies weak pseudorandomness if for every PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$,*

$$\mathbf{Adv}^{\mathsf{PRF\text{-}Psd}}_{\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{PRF\text{-}Psd}}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

*where the experiment $\mathbf{Exp}^{\mathsf{PRF\text{-}Psd}}_{\mathcal{A}}(\lambda)$ is defined in **Fig.** 7.*

$\mathsf{pp} \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
$K \leftarrow \mathsf{PRF.KeyGen}(\mathsf{pp})$.
$(X_0, X_1, st) \leftarrow \mathcal{A}_0^{\mathsf{PRF.Eval}(\mathsf{pp}, K, \cdot)}(\mathsf{pp})$.
if $X_0 \in Q \ \vee \ X_1 \in Q$ **then**
  return 0.
$Y^\star \leftarrow \mathsf{PRF.Eval}(\mathsf{pp}, K, X_b)$.
$b' \leftarrow \mathcal{A}_1(\mathsf{pp}, X_0, X_1, Y^\star, st)$.
return b'.

Fig. 7: Experiment $\mathbf{Exp}^{\mathsf{PRF-Psd}}_{\mathcal{A}}(\lambda)$. In the experiment, Q is the list of all inputs queried by $\mathcal{A}$ to $\mathsf{PRF.Eval}(\mathsf{pp}, K, \cdot)$.

Collision Resistance. Additionally, our construction requires the PRF to be collision resistant even to an adversary knowing the secret key.

Definition 17 (Collision Resistance). *A PRF is collision resistant if for every PPT adversary $\mathcal{A}$,*

$$\mathbf{Adv}^{\mathsf{PRF\text{-}Col}}_{\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{PRF\text{-}Col}}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

*where the experiment is defined in **Fig.** 8.*

Instantiation. A simple and efficient lattice-based PRF satisfying both weak pseudorandomness and collision resistance can be constructed in the (quantum) random oracle model, following [19]. Let $n, m \in \mathbb{N}$, and let p, q be primes with $q \geq p$. For $y \in \mathbb{Z}_q$, define $\lfloor y \rceil_p = \lfloor (p/q) \cdot y \rceil$. Extend this notation to vectors and matrices. Let $\mathsf{H}_{\mathrm{mat}} : \{0,1\}^* \to \mathbb{Z}_q^{m \times n}$ be a random oracle. The PRF is defined as follows:

- $\mathsf{PRF.Setup}(1^\lambda)$: Choose parameters (n, m, q, p), return $\mathsf{pp} = (\mathsf{H}_{\mathrm{mat}}, n, m, q, p)$.
- $\mathsf{PRF.KeyGen}(\mathsf{pp})$: Sample $\mathbf{K} \xleftarrow{\$} \mathbb{Z}_q^n$ and return $\mathbf{K}$.
- $\mathsf{PRF.Eval}(\mathsf{pp}, \mathbf{K}, X)$: Output $\lfloor \mathsf{H}_{\mathrm{mat}}(X)\mathbf{K} \bmod q \rceil_p$.

$\mathsf{pp} \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
$K \leftarrow \mathsf{PRF.KeyGen}(\mathsf{pp})$.
$(X_0, X_1) \leftarrow \mathcal{A}_0(\mathsf{pp}, K)$.
if $X_0 = X_1$ **then**
  return 0.
if $\mathsf{PRF.Eval}(\mathsf{pp}, K, X_0) = \mathsf{PRF.Eval}(\mathsf{pp}, K, X_1)$ **then**
  return 1.
return 0.

Fig. 8: Experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathsf{PRF-Col}}(\lambda)$.

Following [19], this PRF is pseudorandom in the (quantum) random oracle model under the LWE assumption for parameters $n = \mathsf{poly}(\lambda)$, $m = \Omega(n \log q)$, and $q/p = B\sqrt{m}n^{\omega(1)}$, where B is the upper bound for the LWE error vectors. We now establish its collision resistance.

Lemma 4. *The above* PRF *is collision resistant in the (quantum) random oracle model.*

Proof. Let P_1 be the event that $\mathbf{K} = 0^n$, and P_2 the event that the adversary wins when $\mathbf{K} \neq 0^n$. Since $P_1 = 1/q^n$, we focus on P_2. For distinct inputs $X_0 \neq X_1$, the random-oracle outputs $\mathsf{H}_{\mathrm{mat}}(X_0)$ and $\mathsf{H}_{\mathrm{mat}}(X_1)$ are independent and uniform in $\mathbb{Z}_q^{m\times n}$. Hence $Y_i = \mathsf{H}_{\mathrm{mat}}(X_i)\mathbf{K} \bmod q$ is uniformly random in $\mathbb{Z}_q^m$ for each i, and Y_0, Y_1 are independent. By Lemma 5, $\Pr[\lfloor Y_0 \rceil_p = \lfloor Y_1 \rceil_p] \leq (2/p)^m$, which is negligible. Thus $\Pr[P_2]$ is negligible, and the total advantage $\mathbf{Adv}_{\mathcal{A}}^{\mathsf{PRF-Col}}(\lambda) \leq 1/q^n + \Pr[P_2]$ is negligible as claimed.

Lemma 5. *For Y_0, Y_1 uniformly sampled from $\mathbb{Z}_q^m$, $\Pr[\lfloor Y_0 \rceil_p = \lfloor Y_1 \rceil_p] \leq (2/p)^m$.*

Proof. Let $\mathcal{X} = [0, \lfloor q/p \rfloor - 1]^m \cup [q - \lfloor q/p \rfloor, q-1]^m$. For each fixed Y_0, the probability that $(Y_1 - Y_0) \bmod q \in \mathcal{X}$ is at most $(2/p)^m$, and averaging over all Y_0 gives $\Pr[(Y_0 - Y_1) \bmod q \in \mathcal{X}] = (2/p)^m$. Hence the rounding values coincide with probability at most $(2/p)^m$, which is negligible for appropriate parameters.

A.4 Lattice Assumptions

Finally, we recall the standard lattice problems underpinning our constructions: SIS, LWE, and LWR.

Definition 18 (Short Integer Solution (SIS)). *Let $m, n, q \in \mathbb{N}$ with $m > n$ and $\beta > 0$. Given a uniform matrix $\mathbf{A} \hookleftarrow U(\mathbb{Z}_q^{n\times m})$, the $\mathsf{SIS}_{m,q,\beta}$ problem is to find a nonzero $\mathbf{x} \in \mathbb{Z}^m$ such that $\mathbf{Ax} = 0 \bmod q$ and $\|\mathbf{x}\| \leq \beta$.*

Definition 19 (Learning With Errors (LWE)). *Let q, α be functions of n. For secret $\mathbf{s} \hookleftarrow U(\mathbb{Z}_q^n)$, define $A_{q,\alpha,\mathbf{s}}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ by sampling $\mathbf{a} \hookleftarrow U(\mathbb{Z}_q^n)$ and $e \hookleftarrow D_{\mathbb{Z},\alpha q}$, and outputting $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s}\rangle + e)$. The $\mathsf{LWE}_{q,\alpha}$ problem is to distinguish this distribution from uniform on $\mathbb{Z}_q^n \times \mathbb{Z}_q$.*

If $q \geq \sqrt{n}\beta$ and $m, \beta \leq \mathsf{poly}(n)$, then lattice problems with factor $\tilde{\mathcal{O}}(\beta\sqrt{n})$ reduce to $\mathsf{SIS}_{m,q,\beta}$ [23, Sec. 9]. If $\alpha q = \Omega(\sqrt{n})$, then lattice problems with factor $\mathcal{O}(\alpha/n)$ reduce [37,10] to $\mathsf{LWE}_{q,\alpha}$.

Definition 20 (Learning With Rounding (LWR) [2]). *Let q, p, m be polynomial functions of n with $q > p \geq 2$ and $m > n$. The* LWR *problem is to distinguish between*

$$\{(\mathbf{A}, \lfloor \mathbf{A}^T\mathbf{s} \rfloor_p) \mid \mathbf{A} \hookleftarrow U(\mathbb{Z}_q^{n\times m}),\ \mathbf{s} \hookleftarrow U(\mathbb{Z}_q^n)\}$$

and

$$\{(\mathbf{A}, \mathbf{y}) \mid \mathbf{A} \hookleftarrow U(\mathbb{Z}_q^{n\times m}),\ \mathbf{y} \hookleftarrow U(\mathbb{Z}_p^m)\}.$$

Banerjee *et al.* [2] proved that LWR is as hard as LWE when the modulus-to-error ratio is super-polynomial. Alwen *et al.* [1] extended this to polynomial moduli when the number of samples is fixed, and Bogdanov *et al.* [7] removed the remaining modulus restrictions. Under such parameters, an LWR-based pseudorandom generator is secure and stretches $\mathbf{s} \in \mathbb{Z}_q^n$ into $\lfloor \mathbf{A}^T\mathbf{s} \rfloor_p \in \mathbb{Z}_p^m$. This PRG naturally implies a PRF in the random oracle model, where the matrix $\mathbf{A}$ serves as a hash of the PRF input.

B Details of the Zero-Knowledge Protocol Underlying the Lattice-Based CLABS Scheme

We describe how all relations involved in a CLABS signature—circuit satisfiability, certificate validity, and conditional linkability—are jointly encoded as a single instance of the abstract lattice relation of Yang et al. [47]. That framework captures relations consisting of both linear and multiplicative constraints of the form

$$\mathbf{A}\mathbf{x} = \mathbf{y} \pmod q, \qquad \forall (h,i,j) \in \mathcal{M} :\ \mathbf{x}[h] = \mathbf{x}[i] \cdot \mathbf{x}[j],$$

and provides an efficient Σ-protocol with computational zero knowledge under the LWE assumption and knowledge soundness under the SIS assumption. Our goal is to define a public instance $(\mathbf{A}_{\mathsf{clabs}}, \mathbf{y}_{\mathsf{clabs}}, \mathcal{M}_{\mathsf{clabs}})$ whose satisfiability corresponds exactly to the existence of a valid witness for the CLABS signing relation described in Section 5.2.

Witness composition. All secret quantities used in the CLABS signing algorithm are concatenated into a single short witness vector $\mathbf{x}_{\mathsf{clabs}}$. It includes: the bit-decomposition of the signer's attribute $\mathbf{x}_{\mathrm{inp}}$; the internal wire values and gate outputs of the Boolean circuit C; the short vectors $(\mathbf{v}_1, \mathbf{v}_2)$ from the JRS certificate; the tag decomposition $\boldsymbol{\vartheta} = \mathbf{v}_{\vartheta}^{+} - \mathbf{v}_{\vartheta}^{-}$; the selector vector $\mathbf{s}$ and the membership bit b; the embedded tokens $\tilde{\mathbf{Y}} = \{\mathbf{y}_{x,\tau_i}\}_{i=1}^{k}$; the public tag $\mathbf{t}$; and auxiliary vectors $(\mathbf{z}, \mathbf{w})$ appearing in the JRS verification equations. This unified witness ensures that all subcomponents of the CLABS semantics are captured as either linear or multiplicative relations between coordinates of $\mathbf{x}_{\mathsf{clabs}}$.

(1) Circuit satisfiability. The Boolean policy circuit C is decomposed gate-by-gate. Each wire variable is required to be Boolean via a multiplicative constraint $u^2 = u$. Every NAND gate with inputs a, b and output z introduces one auxiliary variable $t_g = a \cdot b$. The gate semantics $z = 1 - ab$ then yields one linear row $z + t_g - \mathbf{x}_{\text{one}} = 0$ and one product triple $(t_g, a, b) \in \mathcal{M}_{\mathsf{clabs}}$. The constant wire $\mathbf{x}_{\text{one}}$ and the output of C are fixed by single linear constraints equal to 1. Consequently, $C(x) = 1$ is fully captured by a set of linear and multiplicative rows compatible with Yang et al.'s protocol, enabling zero-knowledge enforcement of arbitrary Boolean signing policies.

(2) JRS signature relation. The validity of the certified attribute is verified within the same proof system. The Jeudy–Roux–Sanders (JRS) signature [26] uses the verification equations

$$\mathbf{A}\mathbf{v}_1 + \mathbf{w} - \mathbf{B}\mathbf{v}_2 - \mathbf{D}\mathbf{m} = \mathbf{u}, \qquad \mathbf{w} = \boldsymbol{\vartheta} \odot \mathbf{G}_{\text{blk}}\mathbf{v}_2,$$

where $\mathbf{G}_{\text{blk}}$ is the block-diagonal gadget matrix. The first equation is linear and directly embedded as rows in $\mathbf{A}_{\mathsf{clabs}}$, while the second contributes d multiplicative triples $(\mathbf{w}[j], \boldsymbol{\vartheta}[j], (\mathbf{G}_{\text{blk}}\mathbf{v}_2)[j])$. To ensure $\boldsymbol{\vartheta}$ is well-formed, we apply bit-splitting $\boldsymbol{\vartheta} = \mathbf{v}_\vartheta^+ - \mathbf{v}_\vartheta^-$ and enforce the constraints $\mathbf{v}_\vartheta^+[j]\mathbf{v}_\vartheta^-[j] = 0$ and $\sum_j (\mathbf{v}_\vartheta^+[j] + \mathbf{v}_\vartheta^-[j]) = w$. These steps embed the entire JRS verification process into the Yang-style arithmetic model and guarantee that the SoK implies possession of a valid authority certificate.

(3) Membership bit and link semantics. To express conditional linkability, the witness includes a one-hot selector $\mathbf{s} \in \{0,1\}^k$ and a membership bit $b \in \{0,1\}$. Linear equations enforce $\sum_i \mathbf{s}[i] = b$, while each coordinate of the tag $\mathbf{t}$ satisfies $\mathbf{t}[j] - \sum_i u_{i,j} = 0$, where each $u_{i,j}$ is linked by a multiplicative constraint $u_{i,j} = \mathbf{s}[i]\tilde{\mathbf{y}}_{x,\tau_i}[j]$. The Boolean constraint $b(b-1) = 0$ ensures proper formation of b. If $b = 1$, exactly one selector $\mathbf{s}[i] = 1$, binding $\mathbf{t}$ to the corresponding certified token $\mathbf{y}_{x,\tau}$ (linkable case); if $b = 0$, all $u_{i,j}$ vanish, forcing $\mathbf{t} = \mathbf{0}$ (unlinkable case). Thus, linkability semantics are expressed entirely by low-degree arithmetic relations integrated into the same lattice framework.

Unified relation and reduction. Stacking all linear equations yields

$$\mathbf{A}_{\mathsf{clabs}} \cdot \mathbf{x}_{\mathsf{clabs}} = \mathbf{y}_{\mathsf{clabs}} \pmod q,$$

while all multiplicative triples form $\mathcal{M}_{\mathsf{clabs}}$. This unified instance satisfies the syntactic requirements of Yang et al.'s model and efficiently encapsulates the three subrelations: (i) the JRS certificate verification, (ii) the circuit satisfiability, and (iii) the membership-bit logic controlling the tag $\mathbf{t}$. Satisfying this relation implies that the signer holds a valid attribute certificate, the policy $C(x) = 1$ holds, and the tag is correctly generated according to whether $\tau \in L_x$.

The underlying Σ-protocol of [47] achieves computational zero knowledge under LWE and knowledge soundness under SIS. By applying the Unruh transformation [44], as discussed in Section 5.3 we obtain a non-interactive SoK in

the QROM that inherits simulatability under LWE and extractability under SIS. From any accepting transcript, an extractor can reconstruct a consistent witness (x, L_x, b), thus determining whether $\tau \in L_x$ and ensuring that the tag $\mathbf{t}$ corresponds to a genuine PRF evaluation (when $b = 1$) or the dummy value $\mathbf{0}$ (when $b = 0$). This reduction establishes the lattice-based SoK as the formal zero-knowledge core of our CLABS construction.